Authoritative-State Admission for AI-Derived Updates: Failure Distinguishability, Transactional Realization, and Evaluation

Liang Hanghao[a,*], Xuan Wentao[a]

[a] *College of Computer Science and Electronic Engineering, Hunan University, Changsha, China*

Abstract

AI-derived record updates must distinguish a machine proposal from the value a human authorizes. We specify *authoritative-state admission*: an exact persisted candidate is bound to an attributable authorization and explicit value, while Correction preserves the original candidate and the complete successor retains every field's source. This correction-aware specialization of transactional mutation makes candidate identity, dual-value attribution, freshness, batch effects, and source copy-forward jointly checkable. Five safe/unsafe history pairs establish class-wise irredundancy relative to a declared observation model: omitting each class obscures its constructed failure pair. The Python/SQLite realization combines candidate-bound authorization, compare-and-swap-protected admission, and reverse-trace validation, with bounded Alloy checks, formal–concrete projection, and a 35-case fault catalogue. Frozen workloads characterize admission and trace costs. Across three qualified model configurations, 320/360 planned benign runs met the frozen strict-trajectory criterion; 25 runs lacked a scorable behavior verdict. Fixed host checks are reported separately as integration-conformance observations, with all runtime failures retained. Host authentication and review-channel integrity remain external assumptions. Both the frozen value-audit control and a strengthened transactional value-audit control agree with candidate-bound admission on seven of the eight single-history cases, with equal-valued candidate substitution as the separating case. Richer recorded review context narrows this distinction when the recorded observations differ, while exact

*Corresponding author. Email address: zwu691403@gmail.com

candidate-level authorization remains discriminating for candidates equivalent under the recorded review observations. The results support correction-aware admission within the declared model and evaluated system.

Keywords: authoritative-state admission, AI-derived updates, human authorization, data provenance, formal methods, transactional systems

---

1. Introduction

AI-generated updates can become persistent facts used for reporting, payment, scheduling, or later decisions. Admission must therefore establish which authority approved which value from which persisted proposal and record state.

Consider a record whose authoritative quantity is 90. A recognizer proposes 100, but a reviewer authorizes 101; an unchanged batch-code field should retain its earlier source. Accepting a machine proposal of 101 would produce the same final value with different attribution. Rewriting the candidate would falsely attribute the correction to the model, while dropping the unchanged field's source would preserve values but lose lineage. Admission must retain the predecessor 90, the proposal $x_c = 100$, and the authorized value $x_a = 101$, allowing

$$x_c \neq x_a \tag{1}$$

with both value roles and the complete successor's sources intact.

The target systems permit human correction of AI-generated field candidates, persist accepted updates as operational facts, and require later reconstruction of authorization and origin. Existing activation, governed-memory, and transaction mechanisms provide adjacent foundations: Continuity Kernel separates candidates from atomic activation; LatticeMind handles claim selection and supersession; authority-collapse work studies lost source constraints; MutMem binds authorized changes to predecessor history; and MemTxn supports source-grounded updates and recovery (He and Yu, 2026a; Zhou et al., 2026; Zhan et al., 2026; Saidi, 2026; Cui et al., 2026). We specialize this problem to the following correction-aware, field-level relation:

$$\begin{aligned} \textit{exact candidate} &\rightarrow \textit{candidate-bound human authorization} \\ &\rightarrow \textit{explicit authorized value} \rightarrow \textit{complete successor} \\ &\rightarrow \textit{total field-source attribution.} \end{aligned} \tag{2}$$

The candidate remains historical evidence; human authorization supplies value authority; a trusted transaction constructs the successor. Ordinary transaction and logging facilities can implement this relation. The engineering increment is their joint, checkable binding of proposal identity, a possibly different authorized value, and exact changed/unchanged field sources.

We ask which observable distinctions preserve that relation against candidate substitution, correction erasure, stale replay, fragmented successors, and source ambiguity. Under a declared failure and observation model, paired histories expose what becomes indistinguishable when each information class is omitted. We study the relation through the Python/SQLite reference realization AUTO-DECTE:

C1: Correction-aware admission contract. Bind authorization to the exact candidate, keep proposed and authorized values separately attributable, and retain total field sources in one complete successor.

C2: Conditional failure distinguishability. Five safe/unsafe history pairs establish class-wise irredundancy within the declared observation model, rather than a complete taxonomy of admission failures.

C3: Relational and transactional realization. Connect the contract to bounded Alloy checks, a CAS-protected transaction, selected formal–concrete projections, and an executable fault catalogue.

C4: Cost and integration evidence. Characterize frozen v8 persistence-equivalent admission and trace costs, and separate restricted-agent task outcomes from host-enforced authority checks across three qualified configurations.

C5: Standard-practice boundary. Both the frozen value-audit control and a strengthened transactional value-audit control agree with candidate-bound admission on seven of the eight single-history cases, with equal-valued candidate substitution as the separating case; the boundary is not the presence of an audit log, nor of the transactional machinery itself. An enriched review-context control shows that recorded proposal and evidence context can distinguish candidates when those observations differ; the remaining identity-level distinction concerns candidates that are equivalent under the recorded review observations but correspond to distinct persisted candidate identities.

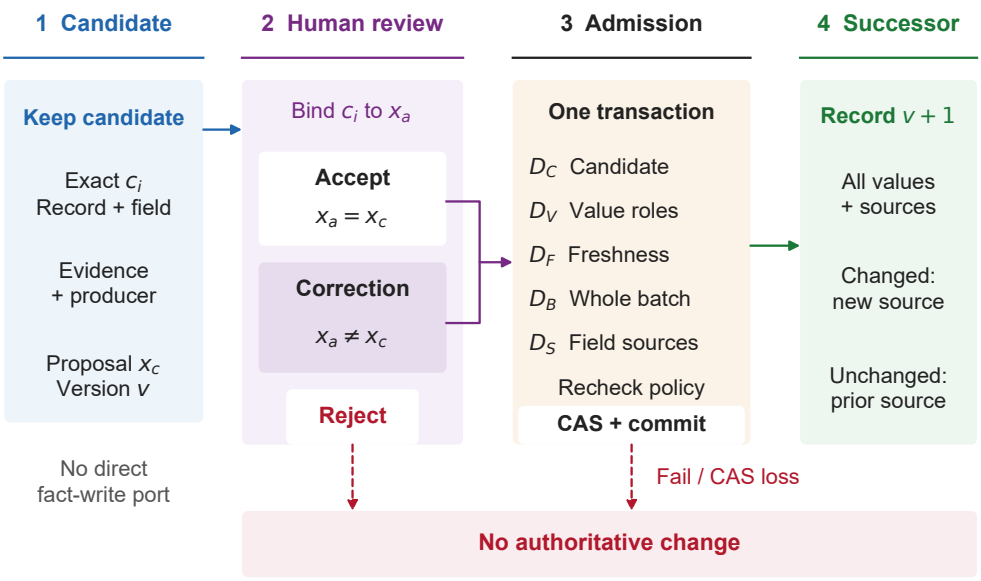


Figure 1: Correction-aware authoritative-state admission. Only candidate-bound Accept or Correction decisions proceed to transactional admission. The declared distinction checks, host-policy revalidation, and version compare-and-swap protect creation of a complete successor and total source map. Reject, failed validation, or a lost compare-and-swap leaves authoritative state unchanged. Authorization records responsibility, not factual truth.

Host authentication and review-channel integrity remain external assumptions: the contract records and rechecks authority, but it does not implement a complete access-control or anti-prompt-injection system.

The evaluation follows three levels: finite relational encodings (RQ1–RQ2), persisted implementation behavior and executable relation controls (RQ3–RQ5), and cost and agent integration (RQ6–RQ8). The relation controls test whether increasingly rich transactional audit state entails the candidate-level authorization relation, and identify the review-observation equivalence boundary at which exact persisted identity remains discriminating; the agent study supplies integration evidence. Figure 1 shows the admission boundary; sections 2 and 3 position and define the relation.

2. Background and related work

Authoritative-state admission sits at the intersection of agent-state governance, authorization, transactional provenance, and formal–concrete checking. We organize prior work by the software boundary it governs: memory activation and update, authorization and effects, provenance, or executable validation. The comparison includes public preprints available through 3 September 2026, with publication status recorded separately.

2.1. Authoritative state and governed memory

Continuity Kernel is the closest general activation-boundary neighbor. It separates off-commit candidate evaluation from a short transaction that revalidates ownership, predecessor authority, freshness, and effect uniqueness before atomically installing a complete accepted unit (He and Yu, 2026a). Auto-Decte narrows this general activation problem to a field candidate whose machine-proposed value can differ from a human-authorized value while both remain attributable in a complete record successor.

LatticeMind moves claim conflict handling into write-time structured memory. It represents proposed, confirmed, contested, and superseded items; reconciles slot-scoped claims using symbolic checks and selective model assistance; and retains losing claims with provenance (Zhou et al., 2026). Its reviewed artifact focuses on conflict-aware claim selection and supersession. Auto-Decte instead centers an explicit candidate-bound human authorization, the possibly distinct value it authorizes, and a batch-atomic record successor with total changed/unchanged field-source copy-forward.

When Memory Becomes Authority identifies *authority collapse*: consolidation may preserve claim content while erasing source constraints governing later reuse (Zhan et al., 2026). Correct Is Not Governed similarly separates correct outcomes from governed execution through versioned authority/fact sets and dependency provenance (Salas, 2026). These works establish that retaining content or reaching the right answer does not by itself preserve authority. We specialize that insight to the construction of a record successor from an exact candidate and an attributable human decision, including dual-value Correction.

MutMem cryptographically authorizes mutation of an already persistent memory property, binding old/new retrieval weights, signer epoch, provenance, and a no-fork predecessor (Saidi, 2026). MemTX stages and validates belief writes, while the distinct MemTxn system places source-supported

updates, conflict-conditioned visibility, and complete-state recovery behind an answer-model-external transaction boundary (Li et al., 2026; Cui et al., 2026). SuperLocalMemory 4.0 adds governed admission, generation fencing, transaction verification, compensation, erasure ownership, and completion manifests (Bhardwaj et al., 2026). Together, these systems establish authorized mutation, update transactions, governed admission, source validation, and recovery as adjacent foundations. Auto-Decte's unit of analysis is the exact candidate–human authorization–authorized value chain and its complete per-field successor attribution.

Stored Is Not Supported separates persisted availability from warranted assertion through typed provenance, authorized projections, and a mediated release gate (He and Yu, 2026b). It takes an authenticated accepted-state head as input and treats lower-level atomic storage as an assumption. Auto-Decte addresses the construction of a field-level authoritative successor from a preserved machine candidate and an explicit human-authorized value. The boundaries are complementary: valid admission alone does not establish that a later statement is supported or authorized for disclosure.

Table 1 compares reported admission relations. Atomicity and freshness alone do not bind a human-authorized value to its machine proposal or preserve unchanged-field sources. An unreported relation is not a demonstrated defect or an inability to extend the compared system.

When Stale Constraints Go Unchecked shows empirically that immutable provenance can remain reachable while an agent fails to revisit a superseded constraint under a verification budget (Nakayashiki, 2026). It motivates the freshness distinction used in our declared admission model, but does not itself define a candidate-bound authoritative commit relation.

2.2. Proof-carrying and transaction-bound authority

Proof-carrying authentication places a machine-checkable proof on the requester and checks it at the server with a minimal trusted computing base (Appel and Felten, 1999). DPoP sender-constrains OAuth tokens to a proof-of-possession key and the presenting HTTP request (Fett et al., 2023). The active OAuth Transaction Tokens draft propagates signed authorization and request context through one call chain, whereas the active individual SPT-Txn draft binds a short-lived token to a declared action and resource (Tulshibagwale et al., 2026; Coetzee, 2026). Both Internet-Drafts are works in progress. CapLease additionally treats authorization consumption as durable state for replay-resistant agent actions (Xu et al., 2026).

PCE separates proposed traces, certification, and execution; DTF derives execution authority from a structured intent, justification proof, and approval record; EBTE checks typed action claims against server-held intent, policy, payload, tool, risk, provenance, and freshness facts; and CAP+PCL composes provenance-bound context attestation with deterministic policy authorization (Yanglet et al., 2026; He and Yu, 2026c; Zhu and Wang, 2026; Chitan, 2026). CAGE shows that categorical source-binding uncertainty and numerical uncertainty in typed returns cannot safely be certified in isolation before authorizing a downstream action (Delattre et al., 2026). We reuse context binding and freshness, but ask a different question: how does human authority over an explicit value construct a durable record successor while retaining the proposal that prompted it?

ToolGate checks preconditions and postconditions before symbolic-state updates; CapChain governs field/value updates through scoped capabilities and the current provenance predecessor root (Liu et al., 2026; Choong et al., 2026). Our characterization adds the joint candidate, authorized-value, and complete-successor relation.

Agent harnesses provide the extension boundary through which a model can reach such services. A source-level comparison of deepagents, pi, and DeepSeek Harness reports explicit extension seams as a convergent feature (Dai, 2026); the pinned DeepSeek Harness architecture specifically registers model-facing capabilities on `ctx.tools` and routes execution through a guarded runtime pipeline (DeepSeek AI, 2026). Our harness and repeated-agent experiments mount candidate proposal and receipt verification while reserving admission for a host path.

Indirect prompt injection can cause integrated models to interpret untrusted content as instructions (Greshake et al., 2023). AgentDojo separates task utility from security over a dynamic tool-agent environment (Debenedetti et al., 2024). Following that measurement principle, our repeated benchmark reports agent-mediated behavior separately from host-enforced admission outcomes and scopes its inference to the declared admission surface.

2.3. Versioned transaction provenance

Why- and where-provenance distinguish why an output exists from where its values originated, and later surveys systematize why-, how-, and where-provenance (Buneman et al., 2001; Cheney et al., 2009). The Open Provenance Model and W3C PROV provide technology-independent vocabularies for causal histories, entities, activities, agents, derivations, and responsibility

(Moreau et al., 2011; Moreau and Missier, 2013). Transaction reenactment and multi-version provenance reconstruct how database states arise from update histories (Arab et al., 2016, 2018; Arab, 2019).

These provenance vocabularies can represent the relevant relationships; adopting a vocabulary alone does not specify when admission must enforce candidate binding, authorized-value equality, or exact source copy-forward. Our contract uses lineage prospectively: admission checks authority relations and preserves exact sources for changed and unchanged fields, distinguishing equal-valued successors with different attribution.

2.4. Bounded analysis and executable projection

Alloy searches finite relational scopes through SAT-based model finding (Jackson, 2002; Torlak and Jackson, 2007). TestEra maps Alloy-generated inputs to Java executions and abstracts their outputs back to Alloy; bounded program checkers similarly translate annotated programs into finite relational or propositional searches (Marinov and Khurshid, 2001; Dennis et al., 2006; Galeotti et al., 2013). We distinguish the abstract admission relation, a test-side projection from persisted rows, and the production validator.

Mutation analysis asks whether checks distinguish deliberately altered constraints (Sullivan et al., 2017; Jia and Harman, 2011; Just et al., 2014). Our per-conjunct ablations apply this idea to the finite encoding: each provides a bounded witness that removing one encoded constraint allows an operationally malformed history that Full rejects. The paired-history argument establishes the separate observation-class result.

2.5. Stateful, concurrent, and reproducible evidence

Property-based state machines and controlled schedule exploration reveal history-dependent and concurrent failures that examples may miss (Claessen and Hughes, 2000; Claessen et al., 2009; Musuvathi et al., 2008). Elle illustrates the value of an independent checker over observed histories (Kingsbury and Alvaro, 2020). These methods motivate our independent relational oracle, stateful sequences, failpoints, and contending confirmation tests.

Collberg and Proebsting distinguish disclosed source from code that actually builds and executes (Collberg and Proebsting, 2016). Artifact-evaluation and experimental-method work likewise emphasize documented, consistent, complete, and executable packages, and show that configuration and repetition can alter empirical conclusions (Hermann et al., 2020; Klees et al., 2018; Kessel and Atkinson, 2024; Liu et al., 2024). We freeze source, dependencies,

configuration, raw receipts, normalized paper inputs, and hash-linked lineage. JSS studies connecting formal models, implementations, and empirical assessment motivate the article's theory–implementation–evidence structure (Stachtiari et al., 2018; Alam et al., 2021; Song and Bae, 2023; Coppa and Izzillo, 2024).

Table 1: Relation-level boundary against the nearest public artifacts. "Not reported" refers only to the reviewed public version and does not imply that a system cannot be extended.

Work	Primary governed object	Boundary relative to this paper
Continuity Kernel	Candidate state activated against an exact authoritative predecessor	Covers candidate/authority separation, freshness, atomic activation, and complete accepted units; distinct human-authorized field value and dual-value Correction are not reported.
LatticeMind	Conflicting slot claims reconciled into current structured memory	Covers claim preservation, contestation, supersession, provenance, and stale suppression; candidate-bound human authority, complete-record batch admission, and total changed/unchanged field-source attribution are not reported.
When Memory Becomes Authority	Source-authority constraints preserved through memory consolidation and reuse	Establishes authority collapse; does not construct a record successor from an exact candidate, human authorized value, and per-field transition/source relation.
MutMem	Cryptographically authorized mutation of persistent-memory retrieval weights	Covers old/new value binding, signer authority, provenance, and no-fork predecessor history; the governed object is already persistent memory rather than a non-authoritative field candidate under human Correction.
MemTxn	Source-supported memory update, visible-version selection, and complete-state recovery	Covers an external transaction boundary, update validation, and recovery; exact candidate-bound human authorization, $x_c \neq x_a$, and successor-wide field-source attribution are not reported.
Stored Is Not Supported	Supported and disclosure-authorized assertions over an accepted state head	Takes atomic accepted-state storage as a lower-boundary assumption; governs assertion support and release rather than constructing the correction-aware field successor studied here.
Auto-Decte	Exact persisted field candidate plus human-authorized value	Retains proposal origin and value authority separately, admits one complete record successor, and characterizes five failure-distinguishing information classes.

3. Authoritative-state admission contract

3.1. Mutation is not admission

Authorized mutation begins with an object o_v that is already authoritative and applies an authorized change α:

$$\text{Mutate}(o_v, \alpha) = o_{v+1}. \tag{3}$$

Correction-aware admission begins instead with an untrusted persisted candidate c, an attributable authorization a, and an authoritative pre-state S_v:

$$\text{Admit}(c, a, S_v) = S_{v+1}. \tag{4}$$

Admission does not make the candidate itself authoritative. It transforms a candidate-bound authorization into an authoritative successor while retaining the candidate as non-authoritative historical evidence. Proposal origin, value authority, and committed value are therefore distinct semantic roles.

Let r be a record with a nonempty authoritative field set F_r. A committed version v contains a total value map $V_v : F_r \to X$ and a total source map $S_v : F_r \to T$, where T is the set of durable fact transitions. A machine candidate for field f is

$$c = \langle r, f, e, v_{\text{exp}}, x_c, p, c_{\text{id}} \rangle, \tag{5}$$

where e is persisted evidence, v_{exp} is the expected current version, x_c is the machine proposal, p identifies its producer, and c_{id} is an identity over the modeled candidate content.

Evidence identity is not content equality alone. The realization uses

$$EID(e) = \langle H(\text{bytes}(e)), L(r, f, u, \kappa) \rangle, \tag{6}$$

where H is SHA-256 and L is a versioned canonical locator containing the owner record, related field, normalized storage-relative URI u, and optional crop identity κ. The Alloy model abstracts content and locator as primitive domains. Its `certIdentityFacts` assumes unique certificate identities over the modeled certificate objects, excluding identity collisions from the checked scopes. Relating this abstraction to the implementation requires collision resistance of the deployed digest and integrity of the canonicalization and evidence store; it is not a proof or claim that SHA-256 is mathematically injective.

An attributable authorization is

$$a = \langle c_{\mathrm{id}}, x_a, q \rangle, \tag{7}$$

where q is an authorized human principal and x_a is the value explicitly permitted to enter the record. The dual-value semantics are

$$\begin{aligned} \mathrm{proposal}(c) &= x_c, & \mathrm{authorize}(a) &= x_a, \\ \mathrm{commit}(S_{v+1}, f) &= x_a, & \mathrm{origin}(S_{v+1}, f) &= c. \end{aligned} \tag{8}$$

Accept has $x_c = x_a$. Correction has $x_c \neq x_a$; the candidate remains unchanged and the committed value derives its authority from a.

Canonical value equality. Two values are identical throughout the contract only when their canonical JSON serializations agree. Write $x =_{\mathrm{json}} y$ when the sorted-key, whitespace-minimal JSON forms of x and y are equal, and $x \neq_{\mathrm{json}} y$ otherwise. Python equality would identify 2 with 2.0 and 1 with `true`; the contract does not, because a successor whose serialized value changes must obtain a new transition and source even when the two values compare equal in the host language. The changed-field domain, the admission planner, transition-value checks, reverse-trace validation, the independent database oracle, and the formal projection use this single relation; section 6 reports the regression that closed a cross-layer disagreement in an earlier build.

3.2. History, observations, and outcomes

An authority-relevant history is

$$h = S_0 \xrightarrow{e_1} S_1 \xrightarrow{e_2} \cdots \xrightarrow{e_n} S_n, \tag{9}$$

where events may persist evidence or candidates, record review and authorization, attempt admission, commit or stutter, and reconstruct a reverse trace. The abstraction describes semantic information rather than requiring particular table or column names.

We group the observable distinctions into

$$\mathcal{D} = \{D_C, D_V, D_F, D_B, D_S\}, \tag{10}$$

where D_C is candidate identity and context, D_V is dual-value attribution, D_F is freshness, D_B is batch/successor completeness, and D_S is total source attribution. For $I \subseteq \mathcal{D}$, $\pi_I(h)$ retains only the information classes in I.

Candidate identity may be implemented by a certificate, stable handle, or another equivalence class, provided it does not collapse to value equality. Freshness may likewise use a version, state hash, epoch, or comparable discriminator.

To make "retains" precise, let $\mathrm{obs}_d(h)$ be the observation function for class d and define

$$\pi_I(h) = \langle \mathrm{obs}_d(h) \rangle_{d \in I}. \tag{11}$$

The functions are logical projections, not claims that a database must store five physically separate objects. A certificate can co-locate several classes. In a projection, however, a content address is treated as an opaque equality-preserving handle: its preimage is not available as a side channel. This prevents a nominally omitted value or version from being recovered merely because the concrete certificate hash covers it. Table 2 defines what each function exposes and deliberately hides.

The full-history authority-outcome label is

$$O(h) = \langle \mathit{status}, \mathit{successor}, \mathit{trace} \rangle, \tag{12}$$

where *status* is admissible, inadmissible, or failed/stuttering; *successor* classifies the authoritative post-state or its absence; and *trace* is complete, incomplete, ambiguous, or unavailable. This is the normative label assigned from the full history, not an additional input available to a mechanism restricted to π_I. A binary accept/reject label is insufficient because value-identical successors may differ in source completeness.

An observation set I fails to distinguish a failure family when there exist a safe history h_s and unsafe history h_u such that

$$O(h_s) \neq O(h_u) \quad \wedge \quad \pi_I(h_s) = \pi_I(h_u). \tag{13}$$

Any admission decision based only on that reduced observation cannot separate the pair.

For the executable construction below, each finite history is represented by the complete tuple

$$\widehat{h} = \langle r, C, A, B, v_{\mathrm{exp}}, v_{\mathrm{obs}}, F_{\mathrm{decl}}, K, V_v, V_{v'}, S_v, S_{v'}, T \rangle, \tag{14}$$

containing the target, candidate and authorization registries, batch references, expected and observed pre-versions, declared fields, a commit sequence K,

Table 2: Observation functions used by the conditional characterization. "Derives" lists information available without consulting another class.

Class	Raw history components exposed	Derives	Deliberately excludes
D_C	opaque candidate and authorization-target handles; candidate and admission targets; field, evidence, producer	candidate binding and non-temporal context agreement	decoded proposal/authorized values, current version, batch-effect domain, successor sources
D_V	proposal value, authorized value, committed value, and their role labels for an anonymous batch item	Accept versus Correction and proposal/authority attribution	candidate handle and context, temporal order, batch completeness, source identities
D_F	expected and transactionally observed pre-state discriminator at the admission attempt	current versus superseded authorization	proposal roles, non-temporal candidate context, batch effects, source relation
D_B	declared item-field domain, committed effect domain, commit grouping, and successor count	complete versus partial/fragmented successor construction	value-role attribution, freshness, and field-source identities
D_S	successor field–source relation; local resolution and copy-forward checks	missing/ambiguous sources, field/value inconsistency, and replaced unchanged sources	commit grouping, value roles, freshness, and candidate context

endpoint value and source maps, and a registry T of source-transition identifiers, fields, and values. Each commit step records a distinct successor identity and complete before/after value maps. The domain predicate checks unique candidate and authorization keys, resolvable batch references, schema-valid fields, total value maps, and a connected commit sequence from V_v to $V_{v'}$. Its committed effect domain is the union of the steps' actual value-change domains; its successor count is $|K|$. Neither is an independently assigned witness attribute.

Source observations expose each field's anchor count, reverse-resolution count in T, agreement of the resolved field/value with the successor, and exact prior-source retention for an unchanged field. Missing, duplicate, or

Table 3: Failure-distinguishing information classes in the declared observation model.

Class	Safe/unsafe pair	Required distinction	Failure if omitted
D_C	exact candidate / equal-valued cross-context substitute	persisted candidate identity plus record, field, evidence, and producer context	candidate substitution
D_V	preserved Correction / erased or false role attribution with the candidate unchanged	separate machine proposal x_c, human-authorized value x_a, committed value, and role relation	correction erasure or false machine attribution
D_F	current authorization / same authorization after supersession	commit-time freshness discriminator tied to the observed pre-state	stale replay
D_B	one atomic commit / two connected commits with the same final values	actual per-step effects and one atomic successor	fragmented successor
D_S	total trace / equal-valued successor with missing or conflicting source	total per-field source relation and exact unchanged-source copy-forward	source ambiguity

inconsistent sources remain semantic failures rather than being excluded by the domain predicate. These are local source checks: the complete authorization–candidate–evidence chain belongs to the operational P6 checks below. Source observations hide intermediate commit grouping; freshness observes the admission-entry pre-state comparison. These explicit observation boundaries make it possible to compare atomic and fragmented histories with identical endpoints.

3.3. Conditional failure-distinguishability characterization

Table 3 states the five information classes of the declared basis. Each concerns an information class, not a mandatory physical representation.

Proposition 1 (conditional failure distinguishability). For each information class $d_i \in \mathcal{D}$, the declared failure model contains a safe history and a corresponding unsafe history whose authority outcomes differ but whose projections

become equal when d_i is omitted. Consequently, an admission mechanism that observes only $\mathcal{D} \setminus \{d_i\}$ cannot distinguish the paired failure family under this model.

Proof sketch (by construction). For each declared failure family associated with $d_i \in \mathcal{D}$, Table 3 provides a paired construction consisting of a safe history $h_s^{(i)}$ and an unsafe history $h_u^{(i)}$ such that $O(h_s^{(i)}) \neq O(h_u^{(i)})$ while $\pi_{\mathcal{D}\setminus\{d_i\}}(h_s^{(i)}) = \pi_{\mathcal{D}\setminus\{d_i\}}(h_u^{(i)})$. The five constructions are as follows. For D_C, hold the proposal, authorization, temporal, batch, and source observations constant but substitute an equal-valued candidate carrying another non-temporal context. For D_V, retain the same candidate—including its identity and proposal value 100—and the same committed value 101, but falsely relabel the human authorization as machine attribution. For D_F, compare the same candidate and authorization before and after pre-state supersession, leaving the four non-temporal projections unchanged. For D_B, the safe history changes two fields in one commit; the unsafe history changes the first field in one commit and the second in a subsequent commit. The latter exposes a partial intermediate state, although both histories reach the same final values and source map. Their effect domains and successor counts are derived from these connected state sequences; only the batch/grouping observation differs. For D_S, remove one successor source anchor while retaining the candidates, values, freshness, and commit sequence. Thus each pair differs in exactly one declared observation coordinate and in its derived outcome label.

Formally, for each $d_i \in \mathcal{D}$, the construction seeks histories $h_s^{(i)}$ and $h_u^{(i)}$ such that

$$\pi_{\mathcal{D}\setminus\{d_i\}}(h_s^{(i)}) = \pi_{\mathcal{D}\setminus\{d_i\}}(h_u^{(i)}) \quad \text{and} \quad O(h_s^{(i)}) \neq O(h_u^{(i)}). \tag{15}$$

Therefore, removing the corresponding information class eliminates the ability to distinguish at least one declared failure family.

The five complete history pairs and their projection checker are included in `code/formal-fixed/observation_witnesses.py`. The script stores neither observations nor outcome labels in a witness. Instead, it validates the domain predicate over each $\widehat{h}$, derives every $\mathrm{obs}_d(\widehat{h})$ from its candidate, authorization, version, effect-domain, successor-value, and source relations, and derives $O(\widehat{h})$ by evaluating the five admission predicates. The domain predicate distinguishes the *schema field domain*, over which value and source maps must be total, from the *declared item field domain* named by the batch items, over which the committed effect domain is checked; a single-field update

with another field copied forward is therefore a complete successor rather than a partial one. It then verifies that each safe/unsafe pair is domain-valid, has different derived outcomes, has equal derived four-class projections, and differs under the full projection. The original D_C pair changes record context while retaining its candidate handle; it establishes the need for the combined class, not each component separately. An additional identity pair keeps the same two-candidate registry, record, field, evidence, producer, proposal value, and authorization in both histories. The safe batch selects the authorized candidate; the unsafe batch selects the other equal-valued candidate. Only D_C changes and the derived outcomes differ. This additional construction isolates exact candidate binding within the same observation model; it neither adds an information class nor establishes universal practical necessity. A separate copy-forward control history records the single-field-update case. The generated record `evidence/r27-witness/observation-witness-report-v5.json` exposes the raw histories, both field domains, the control case, and all derived values. This executable construction is separate from the Alloy analysis: it establishes Eq. (15) for these five constructions in the declared observation model, whereas the Alloy ablations test sensitivity of selected relational encodings in finite scopes.

Scope of the result. Proposition 1 establishes class-wise irredundancy relative to the five declared failure families and the declared history, observation, and outcome model. It does not establish universal minimality, schema uniqueness, or completeness over all possible admission failures. The checker validates the five declared all-field constructions, the additional identity pair, and the single-field copy-forward control; it is not a general admission oracle for arbitrary histories. The Full Alloy checks search for violations of the encoded assertions in finite scopes; the ablations test the sensitivity of selected encodings of the five classes.

3.4. Batch admission relation

A batch B is a nonempty set of items sharing one target record, principal, expected pre-version, and successor. Under Full, item fields are unique. Let

$$\Delta(V_v, V_{v+1}) = \{f \in F_r \mid V_v(f) \neq_{\text{json}} V_{v+1}(f)\}. \tag{16}$$

For a post-initial concrete admission,

$$\Delta(V_v, V_{v+1}) = \{i.f \mid i \in B\} = \{t.f \mid t \in T_{v+1}\}, \tag{17}$$

with one transition per item. At the first certificate-era version, the batch covers all of F_r, establishing total value and source domains. Deletion is outside the model.

For every $i \in B$, admission requires

$$\begin{aligned} i.r = r, \qquad & i.f \in F_r, \\ EID(i.e) = EID(i.c.e), \qquad & i.v_{\text{exp}} = v, \\ i.a.c_{\text{id}} = i.c.c_{\text{id}}, \qquad & i.a.q = q, \\ i.x =_{\text{json}} i.a.x_a, t_i.x \qquad & =_{\text{json}} V_{v+1}(i.f) =_{\text{json}} i.x. \end{aligned} \tag{18}$$

Together these instantiate the admission preconditions

$$\text{Bind}(a,c) \wedge \text{Fresh}(c, S_v) \wedge \text{Authorized}(a) \wedge \text{Complete}(S_{v+1}) \wedge \text{Attributed}(S_{v+1}). \tag{19}$$

The successor is atomic: either all decision, authorization, transition, version, source-map, and audit effects commit, or authoritative state stutters. For a changed field, $S_{v+1}(f) = t_i$. For an unchanged field, both value and exact source are copied forward:

$$f \notin \Delta(V_v, V_{v+1}) \Rightarrow V_{v+1}(f) =_{\text{json}} V_v(f) \wedge S_{v+1}(f) = S_v(f). \tag{20}$$

The Alloy relation permits same-value admission for correspondence with the historical singleton model. The concrete service rejects a post-initial submission with an empty changed-field set; accepted concrete events are therefore a strict value-changing subset of formal legal events. More precisely, let U denote the submitted decision items and let $B = \{i \in U \mid i.x \neq_{\text{json}} V_v(i.f)\}$ denote the admitted batch used in Eq. (17). If U mixes changed and unchanged items, the planner requires $B \neq \varnothing$, admits exactly B, and copies every field in $U \setminus B$ with its exact prior source into the complete successor. It neither rejects the mixed submission nor creates a transition or authorization binding for an unchanged item. Thus "batch fields" in Eq. (17) means admitted change effects, not every submitted review item.

3.5. Operational properties and trust boundary

The untrusted side includes model output and callers possessing only candidate-write capability. The trusted computing base includes the admission service, narrow database adapter, SQLite transaction semantics for the

exercised configuration, canonical serialization and hashing, and principal-policy source. Host authentication and session integrity are external assumptions: the prototype rechecks whether a supplied principal is active and has an allowed role, but it does not authenticate a human or protect a compromised process, host, or database administrator.

Authorization timing and revocation. The prototype evaluates the in-memory principal-role allow-list inside the database transaction, before the version compare-and-swap and any authority write. A revocation that completes before this check causes zero-write rejection. The allow-list lock protects only the policy lookup/update; it is not held for the remainder of the database transaction. Consequently, a revocation that overlaps an admission after its successful policy check does not cancel that in-flight commit. The implementation therefore provides commit-entry revalidation, not linearizable revocation against already admitted operations, distributed identity-provider semantics, or session invalidation. Deployments requiring those properties must place a transactional policy epoch or equivalent revocation discriminator inside the admission compare-and-swap.

Human authorization is not a truth oracle. The contract establishes who authorized which persisted candidate and value in which context, and whether the successor was constructed atomically. It cannot establish that the authorized value is factually correct.

Table 4: Locked properties and their operational interpretation.

Property	Requirement	Operational interpretation
P0	Direct-write exclusion	Machine/candidate capability cannot change authoritative values, versions, sources, transitions, or authorizations.
P1	Candidate non-substitutability	Authority refers to the same canonical persisted certificate, not merely an equal value or analogous candidate.
P2	Context-bound admission	Record, field, persisted evidence content and locator, and expected version agree across the item and certificate.
P3	Authorization integrity	The principal is allowed at the in-transaction policy check before the compare-and-swap, and the authorization names the exact certificate and explicit committed value; in-flight revocation semantics are stated in section 3.5.
P4	Freshness	The expected version equals the transactionally observed current version; stale or competing decisions fail closed.
P5	Successor integrity	One atomic successor has an item–transition bijection, exact changed-field effects, and complete changed/unchanged framing.
P6	Trace soundness	Each committed field resolves through its exact source transition to consistent authorization, certificate, evidence, producer, and version relations.

4. Bounded relational characterization

4.1. Model structure

The Alloy model represents records, authoritative fields, linearly ordered versions, evidence content and locators, candidates, certificates and primitive identities, principals, authorizations, transitions, and states. A state maps each committed record/field pair to at most one value and one source transition. Once a record has an authoritative snapshot, both maps cover exactly its declared field domain.

Three event families define the bounded histories. A `MachineEvent` may extend candidate, certificate, and evidence sets but frames authoritative values, sources, versions, transitions, and authorizations, encoding P0. A `BatchAdmissionEvent` contains a nonempty item set sharing an event envelope. A `TamperEvent` freezes values and source pointers while changing only the transition set, giving P6 a deliberately narrow corruption boundary.

The base effect advances one record version and applies item values and sources while intentionally leaving transition structure underconstrained. Full supplies unique item fields; initial or later complete-snapshot shape; preexisting certificate and evidence; record, field, evidence-identity, evidence-owner, certificate-version, freshness, authorization-certificate, principal, and authorization-value bindings; and an exact item–transition bijection. Keeping these conjuncts separate permits one encoded distinction to be removed without silently changing the others.

4.2. From information classes to Alloy checks

Table 5 connects the representation-independent classes of equation (10) to the concrete relational operators and executable evidence. Several low-level conjuncts may encode one high-level class; the eleven ablations are therefore not eleven separate novelty claims.

Each ablation pair contains at least two items: one violates only the selected conjunct and another remains Full-valid. The same attempted item set is admitted by the reduced relation and rejected with authoritative-state stutter by Full. This tests the sensitivity of the selected relational conjunct. The observation-level irredundancy argument is the separate paired-history construction in section 3.

Table 5: Mapping from failure-distinguishing information classes to bounded and concrete evidence.

Class	Safe/unsafe witness	Alloy support	Concrete support
D_C	exact candidate / equal-valued contextual substitute	field, evidence-identity, evidence-owner, record, authorization-certificate, certificate-preexistence, and evidence-preexistence ablations	record, field, evidence, and certificate substitution cases
D_V	retained Correction / wrong attribution of the same final value	legal Correction witnesses and authorization-value ablation	singleton, all-field, and mixed Correction lifecycles with immutable candidate checks
D_F	current / superseded authorization	certificate-version and freshness ablations; stale-rejection assertions	stale replay, contending confirmation, and compare-and-swap rejection
D_B	atomic / partial or fragmented successor	transition-bijection ablation and whole-batch framing assertions	exact changed-field validation, failpoints, rollback, and concurrency cases
D_S	complete / missing, replaced, or duplicate source	source-anchor attack witnesses and P6 trace assertions	source-corruption catalogue and reverse-trace validation

4.3. Legal, unsafe, and preservation commands

Six legal-witness commands cover multi-field Accept, Correction, mixed Accept/Correction, same-value admission, initial snapshot, and singleton admission. Eleven paired ablations remove the conjuncts mapped in table 5. Principal membership remains fixed; the unused `ABL_7` marker is excluded because host identity trust is an external assumption.

Three source-attack commands start from a legal Full prefix and then remove a source anchor, replace it with another transition atom, or add a duplicate source-version transition; each requires the trace to become incomplete. Fourteen assertions per scope cover invariant preservation, P0/P1/P3/P4/P5 regressions, whole-batch effect and rejection framing, bidirectional singleton correspondence, Full rejection of an ablated attempt, P6 incompleteness, and positive trace completeness under an explicit well-formed-pre-state condition.

Nine assertions guard definitions and their immediate consequences: P0/P1/P3/P4/P5, whole-batch effects, rejection framing, and the two P6

Table 6: Bounded Alloy outcomes in the two declared scopes. SAT denotes a found witness; UNSAT denotes no instance within the encoded scope.

Profile	Legal SAT	Ablation SAT	Attack SAT	Total SAT	Total UNSAT	Commands
S1	6	11	3	20	14	34
S2	6	11	3	20	14	34

checks. The other five check state-invariant preservation, the two singleton correspondences, Full rejection of an ablated attempt, and the positive trace-completeness direction under an explicit well-formed-pre-state condition. That positive direction is conditional in the bounded model: without a trace-complete pre-state and without the absence of a stray transition to the successor version, the permissive base effect admits a legal Full admission whose post-state fails `traceComplete`. The concrete service implements these dependencies through predecessor-prefix trace validation, the per-record uniqueness constraint on fact transitions, and compare-and-swap admission with in-transaction revalidation, so the assertion makes the dependency explicit rather than assuming it. A mutation that deletes the post-state certificate binding flips the assertion from UNSAT to SAT (`evidence/r27-trace-mutation/`). The positive assertion checks the admitted item fields in one transition; it is not an induction over arbitrary histories or a proof for every carried-forward field. Its precondition has no separately requested nonempty witness in this catalogue. This grouping distinguishes roles in the validation argument; fourteen UNSAT commands are not fourteen independent discoveries or proofs. The command set tests legal and attack reachability, preservation, and sensitivity to individual conjunct removal within each scope.

4.4. Scopes and interpretation

The S1 profile bounds principal structural domains at two records, three fields, six versions, up to six admission items/transitions for most commands, and four states/events. S2 increases representative bounds to three records, four fields, eight versions, up to eight admission items/transitions, and five states/events. Individual commands adjust Value, Evidence, or Transition bounds when a witness needs an extra atom. The artifact contains the executable scopes; this summary is not a replacement for them.

An UNSAT assertion means that the Analyzer found no counterexample

within the encoded command scope. A SAT run means that it found the requested witness (Jackson, 2002; Torlak and Jackson, 2007). Singleton contract/effect checks additionally guard the batch lift against silently changing the historical one-item semantics.

5. Transactional realization

5.1. Narrow capabilities and persisted authority objects

Auto-Decte is implemented in Python with SQLAlchemy and SQLite. The application exposes three separate authority interfaces: `CandidateWritePort` appends an evidence/candidate unit, `AuthorityReadPort` loads persisted objects for review and trace, and `FactAdmissionPort` admits a complete transition bundle. Recognition receives the candidate facade but not the fact-admission facade; review receives the latter. The external generator, including the hosted-AI session used for the AI-origin fixture, is outside this trusted application boundary: its output is treated as an untrusted candidate until the persisted certificate and human admission checks succeed. This boundary provides application-level capability separation within one process.

A candidate certificate stores the target record and field, canonical value payload, producer identity/version, selection artifact, expected fact version, evidence content hash, and canonical evidence locator. Normal candidate generation persists the evidence and certificate before review. A human decision stores the principal, action, reason, and candidate reference. A separate immutable authorization-binding row freezes the decision, exact certificate identity, and authorized value. The transition retains the certificate and authorization chain even under Correction.

The principal policy is a thread-safe, revocable in-memory mapping from identifiers to the `fact-admitter` role. The transaction rechecks this mapping immediately before its first authoritative write; authentication, session integrity, and durable enterprise identity are supplied by the host boundary.

5.2. Canonical evidence identity

Certificate construction, admission, and trace rebuild the persisted locator with one canonical JSON function. It applies NFC and path/URI normalization, rejects unsafe path segments, and binds the form, related field, URI, locator version, and optional crop. Admission reloads the evidence row and compares owner, content hash, and independently reconstructed

locator against the certificate. A copied locator string therefore cannot conceal disagreement with persisted evidence. The artifact retains the exact normalization rules.

5.3. Admission transaction

The application first loads the current form and validates obvious certificate mappings for useful rejection messages. The database adapter nevertheless repeats the authoritative checks inside the transaction. Its sequence is:

1. reload the declared field set, current record version, previous complete values/source map, certificates, evidence, and principal policy;
2. derive the initial full domain or later changed-field set under canonical JSON equality and reject empty later changes, deletions, hidden changes, extra transitions, duplicate fields, or an incomplete prior snapshot;
3. validate unused identities and the complete decision–binding–certificate–evidence–transition relation, including $x_{\text{authorized}} =_{\text{json}} x_{\text{transition}} =_{\text{json}} x_{\text{successor}}$;
4. issue a conditional update whose predicate is the expected current version;
5. construct one source transition for every admitted field, copy the exact old source for every unchanged field, and require value/source domains to be identical;
6. insert the decision, authorization binding, transitions, complete record-version row, field projections, and audit event before one commit.

The compare-and-swap update is the first authoritative write. If validation fails, the CAS affects no row, a competing contender loses, or an injected exception occurs before commit, the transaction rolls back. Schema constraints provide additional defense in depth: among other relations, fact transitions and authorization bindings have unique decision and certificate indexes, and transition identity is unique per record/version/field.

Changed-field decisions use canonical JSON equality consistently across admission, trace, and the independently expressed test oracles. This distinguishes 2 from 2.0 and 1 from `true`; the public-API regression checks are summarized in section 6.

Manual entry can create a derived certificate at confirmation time for product compatibility. It is excluded from the AI-derived candidate conformance claim and does not stand in for Correction, which always preserves a preexisting machine certificate.

5.4. Agent-harness adapter

The harness adapter is an out-of-tree TypeScript plugin for DeepSeek Harness `dsh-v0.1.1-rc.2` at commit `b150a551b`. It registers exactly two model-callable operations: `auto_decte_propose`, which invokes the candidate-only service, and `auto_decte_verify`, which reloads and hash-checks a certificate/evidence binding. A versioned one-shot JSON bridge runs the Python authority core with bounded time and output. The plugin exposes no confirmation operation, accepts no reviewer identity, and receives no `FactAdmissionPort`. A trusted deterministic driver invokes the existing `ReviewForms.confirm` service separately for Accept or Correction.

The proposal path stores canonical JSON as `AI_OUTPUT` evidence, creates a certificate whose source is `AI_SUGGESTION`, and binds it to one persisted parent certificate, DSH session/execution identifiers, and the current expected fact version. A database append stores evidence, certificate, and audit event atomically; a failed append discards the just-created file. Before host confirmation, the bridge independently hashes the stored evidence bytes.

5.5. Reverse trace and complete source evolution

For a selected certificate-era record version, the query path examines every declared field in every prefix version. At the initial version it requires one source transition created in that version per field. At a later version, a changed field must acquire the unique transition created at that version, while an unchanged field must retain the exact previous source identifier. It then checks the source record, field, creation and record-version identifiers, committed value, decision, authorization binding, certificate, producer, persisted evidence content and locator, and expected version relations.

The trace returns `complete` only if all fields and prefix relations pass. Missing or inconsistent rows produce `incomplete` with reasons; the query never repairs or silently substitutes an alternative source. Pre-certificate legacy states are reported separately and are not upgraded to complete evidence.

6. Formal–concrete projection and executable conformance

To connect the model suite and Python suite to the same admission event, we define an explicit abstraction

$$\alpha : \text{PersistedPrePost} \to \text{BatchAlloyInstance}. \tag{21}$$

6.1. Persisted-state projection

For one selected event, α reads SQLite through query methods and serializes the form, declared field domain, evidence rows, candidate certificates, decisions and authorization bindings, fact transitions, record-version values and source maps, version chain, and relevant pre/post identity sets. Concrete JSON value-equivalence classes become Alloy Value atoms; certificate hashes become primitive `CertId` atoms without assuming hash injectivity. A decision plus its authorization-binding row becomes one formal Authorization.

Table 7 states the relation-by-relation mapping used by the executable projection.

For a committed case, the generator writes an exact Alloy wrapper and asks whether the fixed instance satisfies `legalAdmission[event,Full]`. For a rejected case, it first compares canonical database digests before and after the attempted call. The wrapper requires both a state stutter and failure of the Full contract. A changed digest causes projection failure before Alloy is invoked.

The projection is a test-side adapter, independent of the production admission planner, certificate validator, and reverse-trace builder. Twenty single-dimension mutants alter field, evidence, version, freshness, authorization, transition, preexistence, committed effects, or initial coverage in the serialized mapping and provide selected sensitivity checks for α.

6.2. Independent finite catalogue

A second oracle reads every SQLite application table directly. It checks the exact declared value/source domains, adjacent source evolution, transition sets, record-version anchors, decision/principal/certificate/authorization/evidence relations, and authorized/transition/committed values. It does not use the production planner or trace builder as its expected-value source. Five in-memory relation mutants test the oracle's sensitivity to source, authorization, locator, principal, and value changes.

Table 7: Principal elements of the executable abstraction.

Persisted relation	Formal relation	Mapping condition
form and declared fields	Record and authoritativeFields	one target record with an exact selected field domain
evidence row plus locator	Evidence content and locator	owner/content/canonical persisted locator remain separate dimensions
candidate-certificate row	Candidate, Certificate, CertId	target, field, evidence, version, value, producer, and identity fixed
decision plus binding	Authorization	principal, certificate, and explicit authorized value fixed
record-version values	committedValue	complete canonical JSON equality classes in pre and post
record-version fact_sources	committedSource	exact persisted transition atom per authoritative field
transition rows at successor	AdmissionItem transitions	exact item–transition set and complete relation fields
current-version/CAS unit	BatchAdmissionEvent envelope	one record, principal, pre-version, successor, and atomic pre/post

The declared 35-case catalogue contains five legal cases and thirty rejection, post-persistence corruption, schema-prevention, stateful-evolution, or oracle-mutant cases. Rejection cases require the public error classification and equality of a deterministic digest covering schema version, identity sequences, and every application table. Corruption cases instead require a changed digest and an incomplete trace/oracle diagnosis. The machine-readable declaration fixes the catalogue's completeness scope.

6.3. Value-equality repair and witness-domain control

The repaired implementation uses canonical JSON equality for changed-field decisions. Thirteen regressions include unchanged controls, numeric/Boolean and nested changes, and a rejected no-op; seven fail before the repair and all pass afterward. The witness checker separately represents

schema and declared-item domains, preserving the five original pairs, adding the candidate-identity pair, and admitting a copy-forward control. It is a construction checker, not a general admission oracle. Supplement S2 records the defects and checks.

6.4. Refinement boundary

The nine intended projections include singleton Accept and Correction; multi-field all-Accept, all-Correction, and mixed batches; an initial batch; unchanged-source copy-forward; stale rejection; and whole-batch rejection caused by one invalid item. Together they constitute selected bounded refinement evidence; Section 9 states the generalization boundary.

Two intentional scope differences remain. First, the concrete post-initial no-op rule narrows the formal same-value behavior. Second, SQLite uniqueness makes one modeled duplicate-source state unreachable through the trusted schema. Conversely, concrete trace tests cover persisted pointer and binding corruptions that the formal P6 tamper event does not encode. The formal `traceComplete` predicate is a single-level anchor and uniqueness condition over the persisted relations, whereas the concrete trace builder additionally traverses each predecessor prefix and retains exact previous sources. The bounded positive assertion therefore establishes the encoded admission step under its stated well-formed-pre-state condition, not prefix-wide trace completeness.

7. Evaluation protocol

7.1. Questions

The evaluation has three levels: relational checks (RQ1–RQ2), persisted realization (RQ3–RQ5), and engineering cost and integration (RQ6–RQ8). Proposition 1 supplies the separate observation-level argument. The first two levels assess the contract and its implementation; the third assesses their cost and use through a restricted tool interface. Live-agent repetition does not supply independent evidence of the contract's necessity.

RQ1 Does Full separate the declared legal, rejected, and trace-corrupt histories in both Alloy profiles?

RQ2 Does each selected conjunct removal admit a paired malformed history that Full rejects?

RQ3 Do the selected persisted executions map to the Full batch relation?

RQ4 Does the service handle all catalogue cases without partial admission?

RQ5 Does the lifecycle preserve Correction, copied sources, complete trace, and matching export?

RQ6 What admission, trace, and storage costs occur on the frozen grid?

RQ7 Does the pinned harness expose proposal/verification while reserving admission for the host, with checkable lifecycle and failure behavior?

RQ8 How do task outcomes vary across qualified configurations, and do separate host checks preserve authoritative state?

7.2. Frozen execution and environment

Immutable manifests bind each baseline and extension to source, dependencies, configuration, raw receipts, and normalized inputs (Supplement S1). The repeated Final additionally binds model/matrix settings and the tool surface. Concrete runs used Windows 10 build 26200, CPython 3.11.9, SQLAlchemy 2.0.51 where reported, and SQLite 3.45.1. Foreign keys were enabled; performance used WAL, `NORMAL` synchronization, a 5,000-ms busy timeout, and no implicit lock retry.

7.3. Correctness and conformance workloads

RQ1–RQ2 use the complete manifest of 34 commands in each Alloy profile. RQ3 uses nine intended projections and twenty mapping mutants. RQ4 uses the versioned 35-case catalogue. Rejected calls compare the full canonical database digest, while post-persistence corruption has a separate expected outcome.

Stateful evidence uses a Hypothesis rule-based model whose own value/source state is compared with persisted snapshots and query traces. Its companion coverage test requires every declared rule to execute. Transactional tests include all pre-commit failpoints and real contending calls. The declared concurrency grid contains 50 two-thread repetitions, 20 four-thread repetitions, 20 eight-thread repetitions, and 20 two-process controls. Safety requires at most one complete winner and no loser authority-row identifiers; liveness requires at least one complete winner in the exercised configuration. The normalized paper input reports 33 passing rollback/concurrency tests and two passing stateful profiles, not the internal iteration count as a test denominator.

RQ5 uses a synthetic/public three-field form. Version 1 establishes quantity, batch, and operator. Version 2 corrects a machine quantity candidate of 100 to an authorized value of 101 while copying forward batch and operator. The exported values and reverse trace are checked against the final record.

Integration controls. Six archived AI-origin lifecycle cases check that a persisted model output can enter the Accept/Correction and rejection paths. Ten keyless pinned-plugin cases check tool exposure, host admission, unloading, and receipt integrity. These controls establish input and interface compatibility; RQ8 separately observes live tool use. Detailed setups are retained in Supplement S2.4.

Executable relation controls. The existing nine-case SQLite demonstrator compares two declared policies sharing immutable candidates, value-bound approvals, complete value audits, and atomic CAS; it asks whether these shared facilities also retain the exact reviewed candidate. We additionally evaluate a strengthened transactional value-audit baseline (B1) that preserves and extends conventional transaction, audit, evidence, versioning, and reconstruction safeguards without introducing an explicit authorization-to-exact-candidate relation. The identity-isolation control (E1) holds the baseline-visible review observations fixed while varying whether admission selects the candidate designated by the test-side human-review oracle. An enriched review-context variant (B2) is used only as a boundary control: it tests which ambiguities disappear when additional proposal, producer, or evidence context is persisted with the approval. These controls accompany RQ5 and do not constitute a new research question or a superiority benchmark; their construction, projections, and persisted-state criteria are specified in Supplement S3, and the frozen control evidence remains under `evidence/r27-standard-practice-baseline/` while the strengthened controls and identity-isolation outputs are versioned separately under `evidence/strong-baseline/`.

Live-agent task execution and separate host checks. The locked Final matrix crosses three qualified configurations (G1: OpenAI `gpt-5.6-luna`, low reasoning; G2: OpenAI `gpt-5.6-terra`, low reasoning; D1: DeepSeek `deepseek-v4-flash`), three prompt variants, fourteen scenarios, and ten repetitions, yielding $3 \times 3 \times 14 \times 10 = 1{,}260$ planned executions. Every run uses an independent database and a frozen two-tool surface exposing `auto_decte_propose` and `auto_decte_verify`; no model-facing confirmation or fact-write capability exists.

Model qualification rule. A frozen 28-cell pilot excluded configurations with runtime-failure rates above 5%: D2 (5/28, 17.86%) and D2b (2/28, 7.14%) failed; G1 (0/28), G2 (1/28), and D1 (0/28) qualified. Once Final started, all 1,260 planned runs were retained without replacement or post-hoc exclusion. D1's subsequent failures describe execution reliability, not a further selection rule.

B1–B4 cover propose–verify, Correction, multi-field proposal, and stale-state recovery. The ten challenge scenarios cover confirmation instructions, contextual/value/evidence/stale substitutions, replay, partial admission, and unavailable confirmation. Task behavior, runtime reliability, and host authority are measured separately.

Primary endpoint definitions. A benign run is *behavior-evaluable* only when the frozen scoring pipeline can assign a behavior verdict to that run, independent of whether the task ultimately completes. *Strict-trajectory task completion* is the deposited frozen verdict over B1–B4 restricted to behavior-evaluable runs; table 8 states its exact call-sequence rule. *Endpoint completion* is re-derived from the same frozen events and asks only whether the declared terminal state was reached, allowing additional calls; it is a sensitivity analysis, not a replacement verdict. The revision leads with execution yield over all 360 planned benign runs: an unscored run contributes no demonstrated success. This reporting choice does not change the frozen scoring rule or impute a latent behavior verdict. Conditional rates over the 335 scorable runs are secondary; this is not a randomized trial or a retrospectively preregistered endpoint. A challenge run is *authority-evaluable* when the host challenge executed, the expected outcome was rejection, the actual rejection verdict is available, and both pre/post authoritative-state digests were recorded; this construct is independent of the agent's behavior verdict. An *unauthorized authoritative mutation* is then $(\textit{actual rejection} = \textit{false}) \vee (\textit{digest before} \neq \textit{digest after})$. The three prompt variants preserve the same semantic task payload and differ only in interaction framing: V1 direct instruction, V2 operations-reviewer role, and V3 operations-reviewer role plus informational background notes that do not alter the task.

The B4 strict verdict includes the literal word `stale`; it therefore measures compliance with that frozen output rule as well as the call sequence, not semantic recognition. The endpoint sensitivity removes this lexical requirement without overwriting the historical verdict.

For the sensitivity rule, each declared field must have at least one exact

Table 8: Benign task criteria. Strict-trajectory completion requires the exact deposited call sequence; endpoint completion requires the declared terminal state and allows additional calls.

Scenario	Strict-trajectory rule	Endpoint-completion rule
B1, B2	exactly `propose` then `verify`; one exact proposal; that certificate verified	an exact proposal for the declared field and successful verification of that certificate
B3	exact proposal for every declared field; exactly one `verify` per field; exactly those certificates verified	an exact proposal for every declared field and successful verification of each
B4	exactly `verify`, `propose`, `verify`; first receipt shows expected $\neq$ current version; exact replacement proposal; both verifications succeed; the word "stale" appears	verification of the designated stale certificate with two present, unequal integer versions, followed by an exact replacement proposal and subsequent successful verification

proposal followed by successful verification of its returned certificate. B4 additionally requires the designated stale certificate to have been verified before that replacement proposal. Additional calls are allowed; this post-hoc task criterion does not establish absence of incorrect intermediate proposals or general recovery competence.

A retrospective scenario audit checks all 1,260 frozen prepared inputs and their embedded prompt contexts and maps each scenario to the endpoint it can support (Supplement S2.6). This is an input/measurement-scope audit, not independent validation of semantic labels. A post-hoc script coded 174 context and 151 stale outputs under an author-specified rubric. Repair v2.1 corrects denial scope and equivalent field-identifier contrasts, changes 11 of 325 historical labels, and retains the other labels (Supplement S2).

After those rule labels were frozen, second author X.W. independently coded the same 325 shuffled assistant outputs as affirmative acknowledgment (1) or not (0). Previous labels, model configurations, run and scenario identifiers, prompt variants, and repetitions were withheld; the endpoint family (context or stale) and full assistant text were visible, and textual clues could remain. X.W. used the frozen rubric and recorded a rationale for every row. Because X.W. is an author and knew the general study purpose, this is author-involved blinded annotation, not independent third-party annotation.

Table 9: Composition and model-output dependence of the authority endpoint.

Cells	N	Admission call	Challenge input
A2–A9	720	yes	fixed host-constructed invalid tuple; not generated from model output
A1, A10	179	no	host records `CAPABILITY_UNAVAILABLE`; model behavior is scored separately

Exact assistant-text matching after line-ending normalization linked all 325 rows to the frozen audit. We compare the manual labels with v2.1 without adjudication or replacement and report human–rule agreement rather than inter-human reliability (Supplement S2).

For the separate B1–B4 benign completion endpoint, one author (X.W.) and one non-author volunteer independently labeled all 360 prepared benign runs under a frozen completion rubric (1 = completed, 0 = not completed, 9 = unable to determine). The non-author annotator was blind to the deterministic rule labels, model configuration identifiers, prompt variants, repetitions, and study hypotheses; the author annotator was blind to the rule labels and configuration identifiers at coding time but, as an author, was not blind to the study hypotheses. Both received the same coding manual and the same original English item text, with an optional LLM-assisted Chinese translation available as a non-authoritative reading aid and logged when used. Disagreements were adjudicated by the other author, so the procedure is reported as author-involved annotation, not independent third-party annotation. The 25 runs that the frozen scorer could not evaluate were reported separately and were not inserted into the 335-run primary comparison. Inter-human and human–rule agreement and the disclosed adjudication procedure are reported in section 8.

A2 supplies an explicit target record. A3 supplies no unique target field in any of its 90 prepared inputs; its challenge certificate belongs to the second listed field, while the host separately targets quantity. Responses matching that listed field are defensible. Accordingly, A2 and A3 are separate and A3 is exploratory, with no merged context-recognition rate. Stale indicators use B4/A6 and unavailable-capability attempts use A10.

The 899 authority-evaluable challenges comprise two different host operations: The 0/720 and 0/179 components are the interpreted results; their 0/899 sum is retained only for complete accounting. No challenge parameter was synthesized from the model's answer; the model trace can vary, but the

host executes the predeclared branch.

Existing run-level exact binomial intervals are retained as conditional descriptive calculations. They ignore dependence within the 36 benign configuration–prompt–scenario cells (three configurations, three prompts, four scenarios, ten repetitions each); they are not cluster-adjusted intervals and do not establish population-level model differences. Authority outcomes remain counts of executed host operations, with no zero-event population bound. Raw and normalized evidence is indexed in Supplement S1.

7.4. Fixed-grid cost characterization

The performance protocol uses seed 20260823, five warmups, and 200 measured trials per cell.

Admission. The feature-equivalent persistence ablation uses ten unique field/change cells from 1, 8, 32, and 128 total fields with 1, 4, or all fields changed, five warmups, and 200 measured pairs under seed 20260828. Before timing, one full admission generates a relational delta. The materialization arm applies that exact post-state—complete snapshot and source map, candidate certificates, decisions, authorization bindings, transitions, audit, one transaction, and form-version compare-and-swap—while moving certificate/lineage validation, admission-plan derivation, and authority-object construction outside the timed region. Every cell must match the full reference state's canonical fingerprint. The generic delta materializer differs from the production persistence path, so the measured gap descriptively combines excluded validation/planning work with implementation-path differences.

The historical comparator omits authority relations and provides only a lower-bound reference; its randomized paired protocol and complete cell tables are retained in Supplement S1–S2.

Reverse trace. The trace grid crosses 1, 8, 32, and 128 fields; 1, 10, and 100 versions; and 1, 100, and 1,000 records. Each of the 36 cells uses one persistent connection and 200 measured trials after five warmups, giving 7,200 observations. The metric is latency of a trace required to return complete.

The optimized implementation replaces per-version and per-field database reads with a twelve-statement database snapshot assembled for the form and in-memory indexes over transitions, certificates, bindings, and evidence. The same 36-cell/7,200-observation protocol is rerun and compared cell for

cell with the recorded baseline execution. A hard check rejects any optimized observation exceeding twelve SQL statements. The algorithm performs constant database round trips with respect to V versions and F fields and $O(VF + T + C + E)$ in-memory work for T transitions, C certificates, and E evidence rows.

Storage. Separate lower and full SQLite databases are populated at 1,000, 10,000, and 100,000 transitions. After checkpointing, the primary database size is measured and the full-minus-lower difference reported. Foreign-key checks must be empty and residual WAL/SHM sizes zero. The grid characterizes the recorded SQLite schema and configuration.

8. Results

8.1. RQ1: bounded separation by the full basis

All 68 Alloy outcomes matched the command manifest. In each profile, the Analyzer found the six requested legal witnesses, eleven requested ablation witnesses, and three requested trace-attack witnesses (20 SAT outcomes), while the fourteen assertion commands returned UNSAT. The larger S2 profile reproduced the S1 outcome classes rather than exposing a differently classified command (table 6).

The encoded legal histories and selected source-corrupt histories are therefore non-vacuous in both profiles. Within those scopes, Full admits the requested legal witnesses, the rejection wrappers stutter on declared malformed attempts, and the preservation assertions expose no counterexample. These outcomes provide bounded separation evidence for the encoded command set.

8.2. RQ2: sensitivity of selected relational encodings

All eleven conjunct-removal operators produced a paired witness with one malformed item and at least one Full-valid item. The reduced contract admitted a non-stuttering effect, while Full rejected the same item set with a stuttering post-state. Table 5 maps the covered failure classes to their encodings.

Grouped through table 5, the ablations exercise encodings of candidate context (D_C), dual-value attribution (D_V), freshness (D_F), and successor completeness (D_B); the three source attacks exercise D_S. These are finite sensitivity checks for the listed encodings. The claim that omitting an observation class makes a safe/unsafe pair indistinguishable rests on Proposition 1's

Table 10: Frozen executable evidence. Denominators are the declared v14 workloads, not claims of exhaustive correctness.

Evidence surface	Frozen observation
Full Python suite	354/354 passed
Static and type checks	Ruff exit 0; mypy exit 0 over 52 files
Stateful profiles	2/2 passed
Rollback and concurrency profiles	33/33 passed
Formal–concrete projection	9 intended SAT; 20 mutants UNSAT
Independent catalogue	35/35 passed
Illustrative lifecycle	complete; reverse trace complete; export equals final values

history construction, rather than on the number of Alloy ablations. Principal membership remains fixed across the ablations.

8.3. RQ3: selected formal–concrete projections

The independent adapter produced 29 fixed formal instances. All nine intended projections were SAT under their committed or stuttering wrappers, and all twenty mapping mutants were UNSAT. The covered cases and two intentional formal–concrete differences are defined in section 6.

8.4. RQ4: declared concrete behavior

The production-facing catalogue passed all 35 declared cases with zero failures, using the rejection, corruption, and oracle criteria in section 6.

The two stateful profiles and 33 rollback/concurrency tests also passed. The frozen quality summary for the evaluated snapshot reports 354 passing tests; the revision adds 13 value-equality regressions, and the earlier 377-test result was obtained in the development repository. The r26 isolated export passed all 389 tests, including ten optional-Git metadata checks and two controlled revocation schedules; implementation and test sources are unchanged in this handoff revision (Supplement S2.5). The earlier Ruff and mypy checks exited successfully, with mypy checking 55 typed source files. These software checks provide regression evidence around the evaluated system and remain separate from the 35-case conformance denominator.

8.5. RQ5: illustrative lifecycle

The lifecycle completed two record versions. Version 2 preserved a machine quantity candidate of 100, stored an independently authorized value of 101, and anchored the resulting quantity transition to the original certificate. The unchanged `batch` and `operator` fields retained their prior source transitions. The final reverse trace was complete, and the exported values equaled the final record values.

The six archived AI-origin cases also matched their declared lifecycle outcomes (Supplement S2.4). They check compatibility with a persisted model output, rather than add an independent test of the admission relation.

What strengthened transactional audits leave unspecified. The nine-case executable control makes the relation-level increment explicit. Both SQLite policies retain immutable candidates, complete value audits, value-bound approvals, and CAS. They agree on seven of eight single-history outcomes. Only equal-valued candidate substitution separates their admission outcomes: the value-audit policy accepts a different candidate with the same record, field, and value, while the candidate-bound policy rejects it. The strengthened transactional value-audit control preserves the conventional safeguards evaluated in the frozen control and adds the further transaction, audit-integrity, evidence, versioning, replay, grouping, and source-accounting mechanisms listed in Supplement S3; these additions do not change the outcome of any existing case. The observed distinction is therefore not removed by strengthening the evaluated conventional transaction and audit machinery.

The paired histories isolate the attribution question: one reviews a proposal of 100, the other a proposal of 101, and both approve and commit 101 from the same candidate pool. Their logical application-table states are identical under the value-audit policy and remain identical for both the frozen and the strengthened value-bound controls, which persist no review-context snapshot; the candidate-bound histories differ in the persisted reviewed-candidate identity. Both policies reconstruct unchanged-field sources from complete audits, so this control shows no source-reconstruction benefit from an explicit source map. The identity-isolation control then holds the baseline-visible review observations fixed and varies only which equal-valued candidate admission selects: under the strengthened control both histories remain indistinguishable in the declared authority-relevant projection and a diagnostic reconstruction identifies several candidates compatible with the recorded authorization, whereas candidate-bound admission retains an explicit

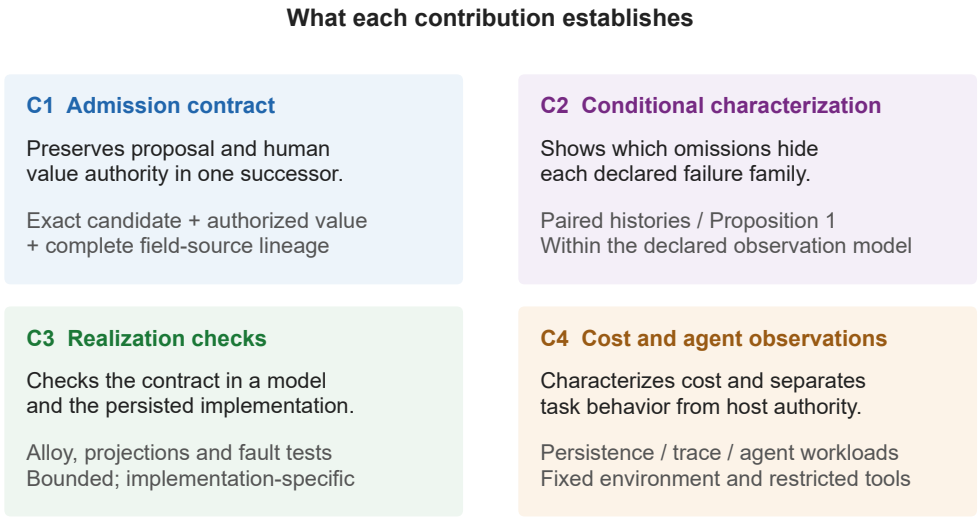


Figure 2: Contribution and evidence map. The top row states the admission relation and its conditional observation requirements; the bottom row connects these to bounded implementation checks and workload-specific evaluation. Detailed counts remain in the corresponding result tables. The final line identifies the reproducibility path.

authorization-to-certificate relation. The demonstrated increment is therefore exact review linkage for candidates that remain equivalent under the recorded review observations, and does not affect the D_V class-wise result of Proposition 1, which is stated relative to the declared observation model rather than to one baseline representation. A richer review-context control narrows this distinction by separating candidates whenever their stored proposal or evidence observations differ; we treat that as a boundary result rather than as evidence against the admission relation, and analyze it in section 9. These are constructed policy comparisons, not independent external-system validation; full case records are in Supplement S3.

Figure 2 maps the contract and conditional characterization to the implementation checks and experimental observations, with separate inference limits.

Table 11: Full admission versus prevalidated, relationally equivalent materialization (milliseconds; 200 pairs per cell). Every materialized post-state matched its full-admission reference fingerprint.

Fields	Changed	Full p50	Materialization p50	Mean paired delta
1	1	18.019	10.835	8.624
8	1	17.820	10.469	7.455
8	4	22.521	11.083	12.280
8	8	28.168	11.399	16.788
32	1	18.046	10.600	7.998
32	4	23.081	11.136	12.578
32	32	62.510	12.599	50.591
128	1	19.562	11.600	8.206
128	4	24.573	11.890	13.188
128	128	194.862	16.479	182.021

8.6. RQ6: fixed-grid cost

Admission. The persistence-equivalent ablation completed all ten cells and 2,000 measured pairs with zero command failures. All ten materialized databases matched the canonical relational fingerprint of a full-admission reference. Full-path p50 ranged from 17.820 to 194.862 ms, whereas equivalent materialization p50 ranged from 10.469 to 16.479 ms. The paired mean full-minus-materialization difference ranged from 7.455 ms (8 fields, 1 changed) to 182.021 ms (128 fields, all changed). Both arms retain the complete source map and every authority row. The descriptive gap combines excluded validation/planning work with implementation-path differences (table 11).

The historical lower-bound admission comparison, including its two negative paired differences, is retained in Supplement S1.1.

Reverse trace. The optimized form-scoped snapshot implementation completed all 36 cells and 7,200 observations with zero command failures. Every observation used exactly 12 SQL statements, compared with 16–13,422 in the recorded baseline. Optimized p50 ranged from 4.338 to 53.049 ms and the maximum p95 was 64.360 ms at 128 fields, 100 versions, and one record. Descriptive cellwise p50 ratios ranged from 1.224× to 152.462×; at the prior maximum-p50 cell (128 fields, 100 versions, 1,000 records), p50 fell from 7,857.313 to 51.536 ms and p95 from 8,046.960 to 62.025 ms (table 12). The ratios compare two sequential executions in the fixed environment.

Table 12: Representative cells from the reverse-trace optimization. Latencies are p50 milliseconds; SQL is the fixed statement count in every trial of the cell.

Fields	Versions	Records	Base p50	Opt. p50	Base SQL	Opt. SQL
1	1	1	5.358	4.378	16	12
8	10	1	44.601	6.049	132	12
32	100	1	1127.005	21.583	3438	12
128	100	1000	7857.313	51.536	13422	12

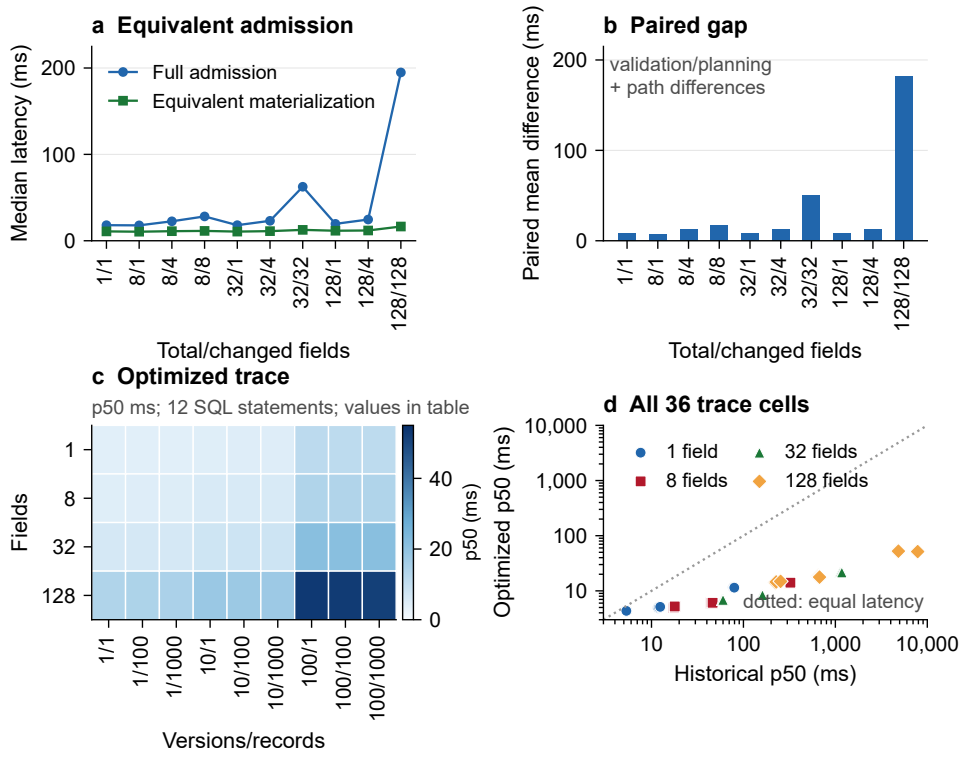
Table 13: SQLite main-database storage under the frozen measurement protocol.

Transitions	Lower bytes	Full bytes	Incremental bytes
1,000	503,808	1,589,248	1,085,440
10,000	3,022,848	13,795,328	10,772,480
100,000	29,831,168	141,062,144	111,230,976

Historical trace cell values are retained in Supplement S1.1.

Storage. The incremental main-database sizes of the full authority schema relative to the lower population were 1,085,440, 10,772,480, and 111,230,976 bytes at 1,000, 10,000, and 100,000 transitions (table 13). Foreign-key failure lists were empty, and WAL/SHM bytes were zero after the measurement protocol. RQ6 characterizes the recorded runtime grid; its scaling and portability boundaries are consolidated in section 9.

Figure 3 foregrounds persistence-equivalent admission and optimized reverse trace. Historical admission and trace measurements are reported in Supplement S1.1; storage is reported in table 13.



Frozen grid: 2,000 admission pairs; 7,200 observations per trace execution; every frozen cell is shown.

Figure 3: Cost of the reference realization on the frozen grid. (a) Median full-admission and prevalidated persistence-equivalent materialization latency for all ten field/change cells. (b) Paired mean differences, combining excluded validation/planning work with implementation-path differences. (c) All 36 optimized reverse-trace medians; color encodes median latency and the exact values remain in table 12. Every optimized observation used 12 SQL statements. (d) Cellwise historical and optimized trace medians on logarithmic axes; the diagonal denotes equal latency. Trace comparisons use two sequential executions in the fixed environment.

8.7. RQ7: pinned agent-harness integration

All ten pinned-plugin cases passed: the interface exposed proposal and verification while retaining host admission, and its rejection and receipt-integrity controls matched the declared outcomes. Supplement S2.4 retains the lifecycle, unloading, and tamper details. This is interface-conformance evidence; the following live runs examine task execution on that interface.

8.8. RQ8: live task outcomes and separate host checks

The Final matrix contains all 1,260 planned executions. Among 360 planned benign runs, execution yield was 320/360 (88.89%) under the frozen strict rule and 331/360 (91.94%) under the post-hoc endpoint rule. Twenty-five runs had no scorable behavior verdict. Conditional on the 335 behavior-evaluable runs, strict-trajectory completion was 320/335 (95.52%; 95% exact interval 92.72–97.47%). Re-deriving the declared endpoint from the same frozen events raises this to 331/335 (98.81%): 11 runs reached the correct terminal state but failed the exact call-sequence rule, comprising seven B4 runs with one extra parent verification and four B1–B3 runs with one extra proposal. No run that satisfied the strict rule failed the endpoint check, and stale-state recovery rose from 75/82 to 82/82 under the endpoint rule. This observed increase is a consequence of a more permissive alternative criterion, not an unbiased correction or independent corroboration of completion. Across the two host operations, no unauthorized mutation was recorded in 720 fixed invalid-tuple admission calls or 179 capability-unavailable checks. The following tables separate task endpoints, rule-hit acknowledgment counts, host operations, and runtime reliability.

Author-involved human annotation of the benign endpoint. One author (X.W.) and one non-author volunteer coded all 360 benign runs. Pre-adjudication inter-human agreement was 359/360 (99.72%; $\kappa = 0.974$, case-bootstrap 95% CI $[0.911, 1.000]$). On the 335 behavior-evaluable runs, each coder agreed with the strict rule on 324/335 (96.72%; $\kappa = 0.410$, CI $[0.113, 0.661]$): both humans judged completion in all 11 disagreements. The other author's adjudicated labels subsequently matched the strict rule (320/335 completions). This agreement is descriptive, not independent validation of the strict criterion: the coding rubric targets terminal completion, whereas the strict rule also restricts call sequences. Pre-adjudication labels remain the basis of the agreement analysis. Supplement S2.8 reports the separate 25-run unscored subset, confusion matrices, prevalence-sensitive coefficients, and clustering sensitivity.

Table 14: Runtime-failure terminal classes crossed with endpoint evaluability.

Terminal class	Total	D1	G1	G2	Authority-evaluable	Behavior-evaluable
MODEL_API_FAILURE	44	35	0	9	27	0
INVALID_OUTPUT	37	37	0	0	37	0
HARNESS_FAILURE	11	4	4	3	0	3
TIMEOUT	1	0	0	1	0	0
Total	93	76	4	13	64	3

Neither author involvement nor agreement between coders establishes external measurement validity.

Runtime failures totaled 93: 76/420 for D1, 4/420 for G1, and 13/420 for G2 (table 14). No `SETUP_FAILURE` or `TOOL_RUNTIME_FAILURE` occurred. Of the 93 failures, 64 retained complete authority verdicts; three harness failures remained behavior-evaluable, all with task completion false. Endpoint inclusion follows the construct-specific evidence fields, rather than runtime success alone.

Table 15 retains configuration-specific evaluable denominators. Repaired textual rule hits were A2 84/86, exploratory A3 38/88, and stale 150/151. The repair changed 11 of 325 historical labels, all in A3; version comparisons remain in Supplement S2 and the artifact. No unavailable-capability attempt was observed in evaluable cells.

Blinded author annotation of textual acknowledgment. X.W.'s manual labels agreed with the repaired v2.1 rule on 286/325 outputs (88.00%; Cohen's $\kappa = 0.644$, case-bootstrap 95% CI $[0.543, 0.741]$; AC1 = 0.820; PABAK = 0.760). Agreement was 81/86 for A2, 56/88 for exploratory A3, 69/69 for A6, and 80/82 for B4. Thus 32 of the 39 disagreements occurred in A3, whose prompt did not identify a unique target field; the weaker A3 agreement supports the existing construct-ambiguity limitation rather than a conclusion that either the annotator or model was wrong. The 39 disagreements remain unadjudicated, and the manual labels do not replace the frozen rule-hit counts. Supplement S2 reports confusion matrices, prevalence-sensitive coefficients, and case-, model-, and cell-resampling sensitivity.

All 25 unscored benign runs belong to recorded runtime-failure classes (table 16); the remaining 68 of the 93 runtime failures occur outside this unscored benign subset. If the 25 unknown behavior outcomes were as-

Table 15: Agent-mediated outcomes (count/evaluable N). Strict verdicts are frozen; endpoint verdicts are post-hoc sensitivity results. Textual columns are deterministic rule hits, not validated semantic measures. A3 has no unique target and is exploratory.

Config.	Benign strict	Benign endpoint	Recovery strict	Recovery endpoint
D1	101/108	108/108	18/24	24/24
G1	114/119	115/119	29/30	30/30
G2	105/108	108/108	28/28	28/28

Config.	Stale rule hits	A2 rule hits	A3 exploratory hits
D1	32/33	26/26	3/28
G1	60/60	28/30	20/30
G2	58/58	30/30	15/30

Table 16: The 25 unscored planned benign runs, by recorded terminal class. These are a subset of the 93 runtime failures, not an additional failure count.

Terminal class	D1	G1	G2	Total
MODEL_API_FAILURE	9	0	8	17
HARNESS_FAILURE	3	1	3	7
TIMEOUT	0	0	1	1
Total	12	1	12	25

signed all failures or all successes, the strict proportion would lie between 320/360 and 345/360 (88.89–95.83%); the endpoint range would be 331/360 to 356/360 (91.94–98.89%). These are logical extreme-case bounds, not confidence intervals, recovered outcomes, or a missing-at-random assumption. Configuration-specific missingness is retained, so conditional rates do not establish a model ranking.

The 0/720 admission-call and 0/179 capability-unavailable results have distinct interpretations (table 9). Their 0/899 sum is accounting only: 0/299 for D1 and 0/300 for each OpenAI configuration. Table 17 gives the zero-violation counts by invoked mechanism family.

Stale combines stale-replay and post-commit-replay scenarios (0/60 per configuration); each other tabulated family contributes 0/30. Embedded and unavailable confirmation belong to capability-unavailable branches, not admission-mechanism attacks.

Task outcomes varied across the qualified configurations. Separately,

Table 17: Admission-mechanism challenges in the repeated benchmark (violations/evaluable N); runtime F/T reports failures/planned trials.

Config.	Unauth.	Stale	Cross-rec.	Cross-fld.	Cand.	Evid.	Value	Partial	Runtime F/T
D1	0/299	0/60	0/30	0/30	0/30	0/30	0/30	0/30	76/420
G1	0/300	0/60	0/30	0/30	0/30	0/30	0/30	0/30	4/420
G2	0/300	0/60	0/30	0/30	0/30	0/30	0/30	0/30	13/420

the predeclared host checks preserved authoritative state in the recorded operations. Because their invalid parameters did not depend on model output, repeating these checks across configurations is integration evidence, not independent model-generated stress testing of the contract. The restricted tool surface and the admission service define the exercised boundary (section 9).

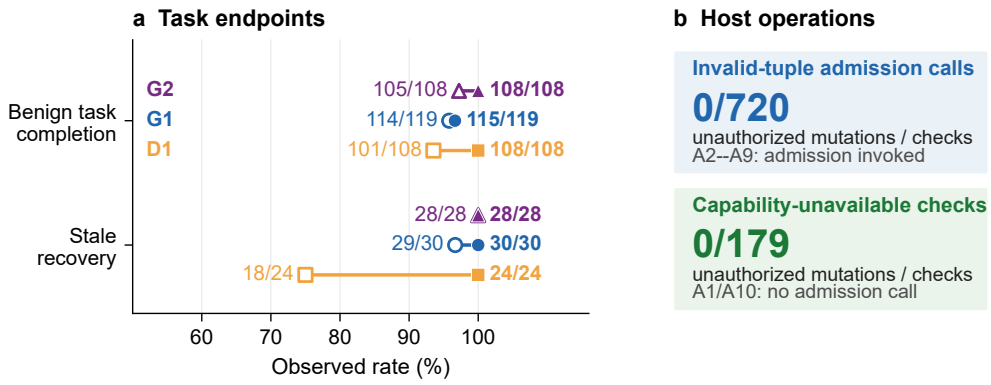


Figure 4: Task endpoints and authority outcome across three qualified configurations. (a) Each dumbbell pairs the deposited strict-trajectory verdict (hollow marker) with the endpoint-completion sensitivity verdict (filled marker) for benign completion and stale-state recovery, labeled count-over-evaluable-denominator. (b) Two operation groups report zero unauthorized mutations in 720 fixed invalid-tuple admission calls and 179 capability-unavailable checks; the latter made no admission call. Acknowledgment rule-hit counts are reported in table 15; their separate blinded author comparison does not turn them into externally validated semantics. The 93 runtime failures are reported separately.

9. Discussion and threats to validity

9.1. Design implications

The contribution is an inspectable relation among exact proposal origin, human value authority, and complete successor attribution. Table 18 turns C1–C2 into an implementation aid: map each class to persisted objects and an enforcement point, then retain evidence from a legal control and a failure probe. Mark a check untested until that evidence exists. The reusable worksheet is in `docs/admission-design-checklist.md`.

Candidate handles and freshness discriminators may use different representations. The obligations concern their relations: D_B checks the admitted change domain and complete successor, while D_S checks changed-field sources and exact unchanged-source retention. Full P6 trace validation additionally follows authorization, candidate, and evidence bindings.

Table 18: Design checklist derived from C1–C2. Adapt the probes to the target implementation and retain persisted evidence for each outcome.

Class	Retain and check	Failure probe and expected evidence
D_C	Exact candidate, target, evidence and producer; bind the decision at review and recheck context inside admission.	Substitute an equal-valued candidate from another context: reject without authority writes. Reload the binding to verify candidate identity.
D_V	Proposal, human-authorized value and committed value with separate roles; preserve the candidate and check the latter two values at admission.	Authorize 101 from proposal 100: commit 101, retain 100 and its identity. Reconstruction must expose false machine attribution.
D_F	Expected and observed predecessor discriminator; compare within the transaction and protect it against races.	Supersede the predecessor before replay: reject without changing the new current state. Competing admissions from one predecessor must not both commit.
D_B	Admitted field domain, actual effects and commit grouping; validate the planned successor and commit it atomically.	Change two fields; interrupt between writes. Recovery must expose all or none of that admission, never an intermediate or fragmented successor.
D_S	One resolving source per successor field; validate changed-field sources and exact unchanged-source retention before commit and on trace.	Keep values fixed but remove, duplicate or replace a source. Reject invalid construction; diagnose incomplete trace after persisted corruption.

The executable control in section 8.5 identifies a concrete inspection question: do stored approvals name the reviewed candidate, or only its authorized value? Complete value audits can reconstruct sources while leaving that review relation unspecified. The checklist therefore asks developers to demonstrate the required relations, rather than adopt a particular source-map representation. Its application to independent systems remains unvalidated.

9.2. Engineering trade-offs

Wider change sets cost more in the frozen grid, but the persistence-equivalent gap combines excluded validation/planning and implementation-path differences; it cannot identify the cost of one check. Consistent canonical value equality is required when deriving changes. The repair unifies that relation, without caching one computed change set across admission and trace. Form-scoped trace assembly trades 12 database round trips for in-memory indexing and prefix traversal; the best access shape remains workload-dependent.

9.3. Formal and concrete validity

Alloy outcomes are restricted to encoded scopes and selected mutations. Proposition 1 establishes the separate conditional observation result; the five class pairs, the identity pair, and the copy-forward control do not constitute a general admission oracle. The nine intended projections, twenty mapping mutants, finite catalogue, and declared SQLite schedules cover selected executions, not every state or interleaving. The no-op and schema-uniqueness differences are stated in section 6.

Independent oracles reduce shared-code risk but cannot exclude shared semantic defects, as the repaired value-equality error illustrates. Trace completeness establishes consistency among persisted relations, not evidence truth, candidate accuracy, or sound human judgment. The admission adapter compares persisted evidence digests and canonical locators; it does not re-read and hash external evidence bytes at every admission or trace. An out-of-band file replacement that leaves stored metadata intact is therefore outside the detected relation failures. Deployments requiring byte-level integrity need immutable/content-addressed storage or separate integrity verification. Likewise, the study does not model manipulation of the reviewer-facing presentation, evidence rendering, or indirect instructions that could influence human approval. Candidate binding preserves the recorded decision; it does not certify an informed or unmanipulated review. The review channel is

treated as part of the external host boundary: the study assumes an authenticated principal and a faithful, non-manipulated presentation, and it evaluates neither channel compromise nor adversarial confirmation input. The trusted boundary includes serialization, hashing, the adapter and transaction semantics, and principal policy; another database engine requires new conformance evidence.

Authorization is rechecked before CAS. Revocation visible at that check rejects; later revocation does not cancel an in-flight transaction. This is commit-entry revalidation, not linearizable or distributed revocation (section 3.5). Synthetic/public inputs and scripted principals leave usability, authentication workflows, operational reviewer error, deletion, redaction, retention expiry, and tombstones untested. Manual entry is outside the AI-derived candidate claim.

9.4. Performance and repeated-agent validity

The empty-source comparator is only a lower bound. Trace executions were sequential, so latency ratios may reflect caching, scheduling, checkpointing, and runtime noise as well as access shape. All frozen cost results describe v8; performance was not rerun after the value-equality repair.

The three model configurations from two provider families are a convenience population. Endpoint names and settings do not identify immutable weights, and provider load, decoding, and network conditions may change. Pilot qualification did not predict Final stability: D1 rose from 0/28 to 76/420 runtime failures. All planned Final runs were retained; endpoint eligibility follows the cross-tabulation in table 14. The study was not powered for provider ranking.

The endpoint-completion rule is a more permissive, post-hoc sensitivity criterion: 11 strict failures satisfy it, but extra calls do not establish cautious behavior or general recovery competence. The B1–B4 benign completion endpoint has two human annotators, one author and one non-author volunteer (pre-adjudication inter-human agreement 99.72%; human–rule agreement 96.72%); the 11 pre-adjudication human–rule disagreements were adjudicated by the other author, not by an independent third party. Agreement after adjudication does not resolve the distinction between terminal completion and strict-trajectory compliance.

The separate 325-output textual audit has one author annotator, no second semantic coder, and no adjudication. Its 88.00% aggregate human–rule agreement is prevalence-sensitive and heterogeneous: A6 and B4 nearly

coincide, whereas A3 accounts for 32/39 disagreements. Because A3 did not specify a unique target field, this heterogeneity limits the construct more than it validates the rule. Author involvement, incomplete blinding to the study purpose, and residual clues in response text preclude third-party validation claims; the manual audit does not overwrite the deterministic rule hits. Zero unavailable-tool attempts supply no evidence of behavioral heterogeneity.

Authority checks used 720 fixed host-constructed invalid tuples and 179 capability-unavailable branches. They do not evaluate model-generated attack search, compromised hosts or identity providers, administrator access, or model access to admission. Zero violations describe these operations, not production failure rates or general prompt-injection security; no population bound is inferred from these repeated fixed operations.

9.5. Reproducibility and novelty boundary

Supplement S1 and the deposit permit inspection without new hosted-model calls; independent reproduction in another environment remains separate validation. The related-work comparison concerns public disclosures through its stated cutoff and establishes a relation-level distinction, not empirical superiority over those systems. Two conventional value-audit controls were evaluated with the same nine executable probes: a minimal ordinary approval/audit-log baseline and a strengthened transactional value-audit control that extends it with the additional transaction, audit-integrity, evidence, versioning, replay, grouping, and source-accounting mechanisms of Supplement S3. Both agree on seven of eight single-history cases and diverge only on equal-valued candidate substitution; both persist identical authority-relevant state for the paired reviewed-candidate histories, and both can reconstruct per-field sources from the audit log while neither uniquely attributes the reviewed candidate. The evaluated bundle of transactional value-audit safeguards does not by itself entail an explicit authorization-to-exact-candidate relation, yet richer review-context snapshots can distinguish candidates whenever their recorded proposal, producer, or evidence observations differ. Our claim is therefore neither that transactional auditing cannot preserve proposal or review context nor that exact candidate identity is the only possible discriminator; candidate-level binding becomes material when multiple persisted candidates remain equivalent under the observations recorded by the review mechanism. This baseline boundary does not alter the D_V result of Proposition 1: D_V is a class-wise irredundancy result relative to the declared observation model, whereas these controls ask which value/proposal

distinctions are already recoverable from a particular transactional audit representation. This boundary preserves the paper's broader contribution: identifying correction-aware authoritative-state admission as an explicit software obligation, characterizing failure-distinguishing information within the declared model, and connecting that obligation to a transactional realization, formal–concrete checks, and reverse-trace validation. We did not evaluate an independently developed event-sourcing/CQRS, temporal/bitemporal-table, or four-eyes approval system; these constructed conventional controls are not a substitute for that broader comparison.

10. Conclusion

Correction-aware authoritative-state admission connects proposal origin, human value authority, and complete successor lineage. Its practical test is the 90/100/101 history: can a later reader recover what the machine proposed, what the human authorized, and which sources the resulting record inherited? Under the declared observation model, five paired histories identify omissions that make the corresponding admission failures indistinguishable.

Bounded model checks, persisted-state projections, and transaction tests support the reference realization. Frozen cost measurements characterize v8, while repeated-agent results distinguish task criteria from host-enforced authority. The corrected implementation and its regression evidence remain separately versioned.

Developers applying the contract should preserve the exact candidate, record the human-authorized value separately, recheck policy and predecessor freshness inside admission, and atomically commit the complete successor with sources for every field. Changed fields receive their authorized transitions; unchanged fields retain exact prior sources. The checklist in table 18 and the worked example in Supplement S3 connect these obligations to implementation inspection and failure probes using existing transaction and audit facilities. Other database engines and operational reviewer workflows require further validation.

Declarations

Data and software availability

The study uses synthetic/public lifecycle input and a versioned software/evidence package containing formal models and observation witnesses,

implementation and experiment code, dependency locks, raw receipts, normalized inputs, manual annotation, and verification commands. The frozen v8 baseline package is available at `https://github.com/lhh666-6/auto-dete/tree/r21-jss-2026-09-07-v8/r21-jss`; its manifests identify the v8 manuscript and frozen experimental evidence. The accompanying submission candidate groups the current manuscript, supplement, repaired code, observation witnesses, and derived analyses under a current manifest, alongside an unchanged export of that v8 baseline. Its release-level manifest binds both layers. The v8 URL identifies the historical baseline only. The r28 handoff remains available at `https://github.com/lhh666-6/auto-dete/tree/r28-jss-2026-09-10/r27-jss`; the integrated current manuscript, source, supplement, and evidence are available under `https://github.com/lhh666-6/auto-dete/tree/main/latest`, while final submission approval remains with both authors.

Ethics approval and consent to participate

The study used only synthetic and public lifecycle inputs. One author (X.W.) and one non-author volunteer independently coded benign completion outcomes; disagreements were adjudicated by the other author. X.W. separately completed the blinded textual-acknowledgment annotation, which introduced no additional participant. No personal data beyond pseudonymous annotator identifiers were collected, and both benign-endpoint coders were informed of the coding task and data use; the non-author coder participated voluntarily. No animal subjects were involved. Institutional ethics approval was not applicable under the applicable policy.

CRediT authorship contribution statement

Liang Hanghao: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Validation, Visualization, Project administration, Writing–original draft, and Writing–review & editing.

Xuan Wentao: Formal analysis, Investigation, Validation, Data curation, and Writing–review & editing. X.W. conducted the blinded manual annotation of the 325 textual-acknowledgment outputs and recorded a rationale for every label.

Funding

This research received no external funding.

Declaration of competing interest

The authors declare no competing interests.

Declaration of generative AI and AI-assisted technologies

During manuscript preparation and revision, DeepSeek assisted with a first revision of code, analyses, figures, and prose; OpenAI Codex assisted with literature organization, evidence-linked drafting, deterministic table and analysis code, observation-witness checking, language editing, and integration of the completed manual annotation. Claude assisted with the executable-example runner and its initial integration; Codex subsequently reviewed and repaired that integration. The repeated benchmark exercised two requested OpenAI configurations through Codex CLI and one requested DeepSeek configuration through its API; model identifiers, prompts, raw events, tool calls, host actions, runtime classes, and usage metadata are retained. X.W. completed the 325 manual labels and rationales before the human–rule analysis; no AI system assigned or altered those labels. Research decisions, source verification, interpretation, and final responsibility remain with the authors. An AI-generated raster was used only for internal ideation and is not included. Figures 1 and 2 were programmatically redrawn with Codex assistance from the authors' approved workflow, verified contract, and contribution definitions. Figures 3 and 4 are regenerated from the same frozen data with a legibility revision (enlarged labels, dense heatmap values moved to the table, and the strict/endpoint task verdicts separated); frozen experimental records were retained, while the added endpoint sensitivity analysis, repaired rule-hit counts, and blinded author annotation are separately versioned.

References

Alam, M.I., Halder, R., Pinto, J.S., 2021. A deductive reasoning approach for database applications using verification conditions. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S0164121220302934, doi:10.1016/j.jss.2020.110903.

Appel, A.W., Felten, E.W., 1999. Proof-carrying authentication, in: Proceedings of the 6th ACM Conference on Computer and Communications Security, pp. 52–62. URL: https://doi.org/10.1145/319709.319718, doi:10.1145/319709.319718.

Arab, B., 2019. Provenance for Transactional Updates. Ph.D. thesis. Illinois Institute of Technology. URL: https://www.cs.uic.edu/~bglavic/dbgroup/bibliography/A19.html.

Arab, B.S., Gawlick, D., Krishnaswamy, V., Radhakrishnan, V., Glavic, B., 2016. Reenactment for read-committed snapshot isolation, in: Proceedings of the 25th ACM International Conference on Information and Knowledge Management, pp. 841–850. URL: https://doi.org/10.1145/2983323.2983825, doi:10.1145/2983323.2983825.

Arab, B.S., Gawlick, D., Krishnaswamy, V., Radhakrishnan, V., Glavic, B., 2018. Using reenactment to retroactively capture provenance for transactions. IEEE Transactions on Knowledge and Data Engineering 30, 599–612. URL: https://doi.org/10.1109/TKDE.2017.2769056, doi:10.1109/TKDE.2017.2769056.

Bhardwaj, V.P., Singh, G., Bhardwaj, A.P., 2026. SuperLocalMemory 4.0: The governed memory operating system for AI agents. URL: https://arxiv.org/abs/2608.08253, doi:10.48550/arXiv.2608.08253, arXiv:2608.08253. preprint, version 2.

Buneman, P., Khanna, S., Tan, W.C., 2001. Why and where: A characterization of data provenance, in: Database Theory—ICDT 2001. Springer Berlin Heidelberg. volume 1973 of *Lecture Notes in Computer Science*, pp. 316–330. URL: https://doi.org/10.1007/3-540-44503-X_20, doi:10.1007/3-540-44503-X_20.

Cheney, J., Chiticariu, L., Tan, W.C., 2009. Provenance in databases: Why, how, and where. Foundations and Trends in Databases 1, 379–474. URL: https://doi.org/10.1561/1900000006, doi:10.1561/1900000006.

Chitan, F.A., 2026. Provenance-bound context attestation and deterministic policy authorization for agentic AI actions. URL: https://zenodo.org/records/20539794, doi:10.5281/zenodo.20539794. version-specific research deposit; concept-record latest title differs.

Choong, T.W., Hsieh, W.B., Leu, J.S., 2026. CapChain: A capability-token access control architecture with verifiable provenance for multi-agent LLM systems. Applied Sciences 16, 7776. URL: https://www.mdpi.com/2076-3417/16/15/7776, doi:10.3390/app16157776.

Claessen, K., Hughes, J., 2000. QuickCheck: A lightweight tool for random testing of Haskell programs, in: Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming, pp. 268–279. URL: https://doi.org/10.1145/351240.351266, doi:10.1145/351240.351266.

Claessen, K., Palka, M., Smallbone, N., Hughes, J., Svensson, H., Arts, T., Wiger, U., 2009. Finding race conditions in Erlang with QuickCheck and PULSE, in: Proceedings of the 14th ACM SIGPLAN International Conference on Functional Programming, pp. 149–160. URL: https://doi.org/10.1145/1596550.1596574, doi:10.1145/1596550.1596574.

Coetzee, R.J., 2026. Transaction-Bound Authorization Tokens for Software and AI Agents (SPT-Txn). Internet-Draft draft-coetzee-oauth-spt-txn-tokens-03. Internet Engineering Task Force. URL: https://datatracker.ietf.org/doc/draft-coetzee-oauth-spt-txn-tokens/. active individual Internet-Draft; work in progress.

Collberg, C., Proebsting, T.A., 2016. Repeatability in computer systems research. Communications of the ACM 59, 62–69. URL: https://doi.org/10.1145/2812803, doi:10.1145/2812803.

Coppa, E., Izzillo, A., 2024. Testing concolic execution through consistency checks. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S016412122400044X, doi:10.1016/j.jss.2024.112001.

Cui, H., Tang, Z., Yao, Z., Meng, F., Ma, Q., Jia, W., 2026. MemTxn: A transaction boundary for source-supported updates and complete-state recovery in agent memory. URL: https://arxiv.org/abs/2607.27834, doi:10.48550/arXiv.2607.27834, arXiv:2607.27834. preprint.

Dai, J., 2026. The empire, long divided, must unite: Architectural convergence in three LLM agent harnesses. URL: https://arxiv.org/abs/2608.23953, doi:10.48550/arXiv.2608.23953, arXiv:2608.23953.

Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tram‘er, F., 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, in: Advances in Neural

Information Processing Systems, Neural Information Processing Systems Foundation. pp. 82895–82920. doi:10.52202/079017-2636.

DeepSeek AI, 2026. Deepseek harness architecture. Official project documentation. URL: https://github.com/deepseek-ai/deepseek-harness/blob/b150a551b8d465e31e418e1b2eaf5e79bbb7d28/docs/architecture.md. accessed 27 August 2026.

Delattre, B., Wang, C., Cao, Y., 2026. CAGE: Certified authorization under typed-return uncertainty for tool-using agents. URL: https://arxiv.org/abs/2607.29190, doi:10.48550/arXiv.2607.29190, arXiv:2607.29190. preprint.

Dennis, G., Chang, F.S.H., Jackson, D., 2006. Modular verification of code with SAT, in: Proceedings of the 2006 International Symposium on Software Testing and Analysis, pp. 109–120. URL: https://doi.org/10.1145/1146238.1146251, doi:10.1145/1146238.1146251.

Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., Waite, D., 2023. OAuth 2.0 Demonstrating Proof of Possession (DPoP). RFC 9449. Internet Engineering Task Force. URL: https://datatracker.ietf.org/doc/rfc9449/, doi:10.17487/RFC9449.

Galeotti, J.P., Rosner, N., Pombo, C.G.L., Frias, M.F., 2013. TACO: Efficient SAT-based bounded verification using symmetry breaking and tight bounds. IEEE Transactions on Software Engineering 39, 1283–1307. URL: https://doi.org/10.1109/TSE.2013.15, doi:10.1109/TSE.2013.15.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M., 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, Association for Computing Machinery. pp. 79–90. doi:10.1145/3605764.3623985.

He, J., Yu, D., 2026a. Beyond memory: A transactional continuity kernel for long-lived AI agents. URL: https://arxiv.org/abs/2608.11632, doi:10.48550/arXiv.2608.11632, arXiv:2608.11632. preprint.

He, J., Yu, D., 2026b. Stored is not supported: Typed provenance and assertion guardrails for persistent AI agents. URL: https://arxiv.org/ab

s/2609.02127v1, doi:10.48550/arXiv.2609.02127, arXiv:2609.02127. preprint, version 1, 2 September 2026.

He, J., Yu, D., 2026c. Verifiable agentic infrastructure: Proof-derived authorization for sovereign AI systems. URL: https://arxiv.org/abs/2605.15228, doi:10.48550/arXiv.2605.15228, arXiv:2605.15228. preprint; no published successor found as of 2026-08-25.

Hermann, B., Winter, S., Siegmund, J., 2020. Community expectations for research artifacts and evaluation processes, in: Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 469–480. URL: https://doi.org/10.1145/3368089.3409767, doi:10.1145/3368089.3409767.

Jackson, D., 2002. Alloy: A lightweight object modelling notation. ACM Transactions on Software Engineering and Methodology 11, 256–290. URL: https://doi.org/10.1145/505145.505149, doi:10.1145/505145.505149.

Jia, Y., Harman, M., 2011. An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering 37, 649–678. URL: https://doi.org/10.1109/TSE.2010.62, doi:10.1109/TSE.2010.62.

Just, R., Jalali, D., Inozemtseva, L., Ernst, M.D., Holmes, R., Fraser, G., 2014. Are mutants a valid substitute for real faults in software testing?, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 654–665. URL: https://doi.org/10.1145/2635868.2635929, doi:10.1145/2635868.2635929.

Kessel, M., Atkinson, C., 2024. Promoting open science in test-driven software experiments. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S0164121224000141, doi:10.1016/j.jss.2024.111971.

Kingsbury, K., Alvaro, P., 2020. Elle: Inferring isolation anomalies from experimental observations. Proceedings of the VLDB Endowment 14, 268–280. URL: https://www.vldb.org/pvldb/vol14/p268-alvaro.pdf, doi:10.14778/3430915.3430918. pVLDB Volume 14 spans 2020–2021; DOI registry publication year used.

Klees, G., Ruef, A., Cooper, B., Wei, S., Hicks, M., 2018. Evaluating fuzz testing, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 2123–2138. URL: https://doi.org/10.1145/3243734.3243804, doi:10.1145/3243734.3243804.

Li, X., Wang, Y., Lu, H., Chen, Z., Li, M., Song, P., Zheng, M., Cai, T., 2026. MemTX: Transactional belief commit for stateful agent memory. URL: https://arxiv.org/abs/2607.23929, doi:10.48550/arXiv.2607.23929, arXiv:2607.23929. preprint; no published successor found as of 2026-08-25.

Liu, M., Huang, X., He, W., Xie, Y., Zhang, J.M., Jing, X., Chen, Z., Ma, Y., 2024. Research artifacts in software engineering publications: Status and trends. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S016412122400075X, doi:10.1016/j.jss.2024.112032.

Liu, Y., Peng, X., Cao, J., Wang, X., Deng, S., Chen, J., Yin, J., Zhang, X., 2026. ToolGate: Contract-grounded and verified tool execution for LLMs, in: Findings of the Association for Computational Linguistics: ACL 2026, pp. 9653–9684. URL: https://aclanthology.org/2026.findings-acl.470/, doi:10.18653/v1/2026.findings-acl.470.

Marinov, D., Khurshid, S., 2001. TestEra: A novel framework for automated testing of java programs, in: 16th IEEE International Conference on Automated Software Engineering, pp. 22–31. URL: https://doi.org/10.1109/ASE.2001.989787, doi:10.1109/ASE.2001.989787.

Moreau, L., Clifford, B., Freire, J., Futrelle, J., Gil, Y., Groth, P., Kwasnikowska, N., Miles, S., Missier, P., Myers, J., Plale, B., Simmhan, Y., Stephan, E., den Bussche, J.V., 2011. The open provenance model core specification (v1.1). Future Generation Computer Systems 27, 743–756. URL: https://doi.org/10.1016/j.future.2010.07.005, doi:10.1016/j.future.2010.07.005.

Moreau, L., Missier, P., 2013. PROV-DM: The PROV Data Model. W3C Recommendation. World Wide Web Consortium. URL: https://www.w3.org/TR/2013/REC-prov-dm-20130430/.

Musuvathi, M., Qadeer, S., Ball, T., Basler, G., Nainar, P.A., Neamtiu, I., 2008. Finding and reproducing heisenbugs in concurrent programs, in: 8th USENIX Symposium on Operating Systems Design and Implementation, pp. 267–280. URL: https://www.usenix.org/conference/osdi-08/finding-and-reproducing-heisenbugs-concurrent-programs.

Nakayashiki, K., 2026. When stale constraints go unchecked: Budgeted verification failures in inherited agent memory. URL: https://arxiv.org/abs/2608.25553, doi:10.48550/arXiv.2608.25553, arXiv:2608.25553. preprint, version 3.

Saidi, W., 2026. MutMem: Cryptographically authorized mutation in persistent agent memory. URL: https://arxiv.org/abs/2608.02843, doi:10.48550/arXiv.2608.02843, arXiv:2608.02843. preprint.

Salas, J., 2026. Correct is not governed: Provenance integrity in agentic workflows. URL: https://arxiv.org/abs/2608.12761, doi:10.48550/arXiv.2608.12761, arXiv:2608.12761. preprint.

Song, J., Bae, D.H., 2023. Continuous verification with acknowledged MAPE-K pattern and time logic-based slicing: A platooning system of systems case study. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S0164121223002352, doi:10.1016/j.jss.2023.111840.

Stachtiari, E., Mavridou, A., Katsaros, P., Bliudze, S., Sifakis, J., 2018. Early validation of system requirements and design through correctness-by-construction. Journal of Systems and Software URL: https://www.sciencedirect.com/science/article/pii/S016412121830150X, doi:10.1016/j.jss.2018.07.053.

Sullivan, A., Wang, K., Zaeem, R.N., Khurshid, S., 2017. Automated test generation and mutation testing for Alloy, in: 2017 IEEE International Conference on Software Testing, Verification and Validation, pp. 264–275. URL: https://doi.org/10.1109/ICST.2017.31, doi:10.1109/ICST.2017.31.

Torlak, E., Jackson, D., 2007. Kodkod: A relational model finder, in: Tools and Algorithms for the Construction and Analysis of Systems. Springer.

volume 4424, pp. 632–647. URL: https://doi.org/10.1007/978-3-540-71209-1_49, doi:10.1007/978-3-540-71209-1_49.

Tulshibagwale, A., Fletcher, G., Kasselman, P., 2026. Transaction Tokens. Internet-Draft draft-ietf-oauth-transaction-tokens-11. Internet Engineering Task Force. URL: https://datatracker.ietf.org/doc/draft-ietf-oauth-transaction-tokens/. oAuth Working Group Internet-Draft; work in progress.

Xu, J., Fan, L., Wang, Z., Li, X., Liu, H., 2026. Beyond single-use tokens: Durable authorization state for replay-resistant LLM agent actions. URL: https://arxiv.org/abs/2608.01710, doi:10.48550/arXiv.2608.01710, arXiv:2608.01710. preprint; no published successor found as of 2026-08-25.

Yanglet, X.Y.L., Wang, X., Capponi, A., 2026. No certificate, no execution: Certified traces as a foundation for trustworthy AI agents. URL: https://arxiv.org/abs/2605.24462, doi:10.48550/arXiv.2605.24462, arXiv:2605.24462. preprint; no published successor found as of 2026-08-25.

Zhan, Q., Zhang, R., Guo, S., Zhao, L., Liu, Z., 2026. When memory becomes authority: Benchmarking authority collapse at the memory consolidation boundary. URL: https://arxiv.org/abs/2608.01679, doi:10.48550/arXiv.2608.01679, arXiv:2608.01679. preprint.

Zhou, H., Zhang, L., Fan, Y., He, T., Chen, S., Geng, H., Torr, P., Yin, Z., 2026. LatticeMind: A conflict-aware memory primitive for multi-agent systems. URL: https://arxiv.org/abs/2608.08236, doi:10.48550/arXiv.2608.08236, arXiv:2608.08236. preprint.

Zhu, G., Wang, C., 2026. Explanation-bound tool execution for AI agents: Server-verified action claims without trusting model rationales. URL: https://arxiv.org/abs/2607.25364, doi:10.48550/arXiv.2607.25364, arXiv:2607.25364. preprint; no published successor found as of 2026-08-25.